

Generative AI & AI Agents

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité
<https://www.youtube.com/@mohamedYoussfi/videos>
<https://github.com/mohamedYoussfi/agentic-ai>



Laboratoire Mathématiques, Sciences de Données
et Technologies Emergentes



SEMINAIRE MDSET Leçon inaugurale

Generative AI & Agentic AI: Vecteurs
incontestables de la transformation digitale

Pr. Mohamed YOUSSEFI

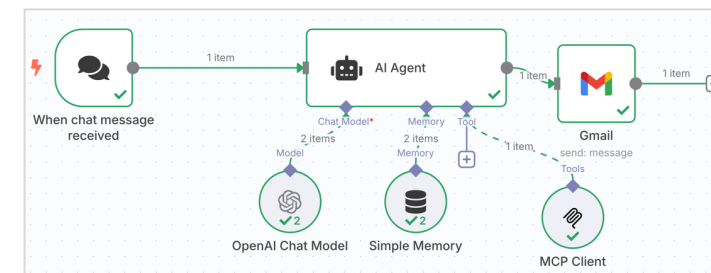
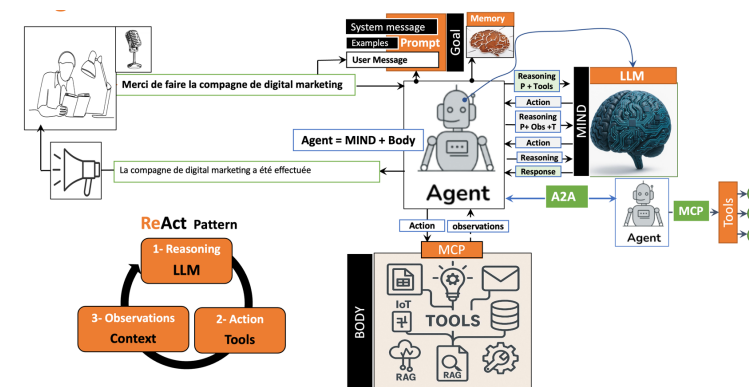
Professeur de l'Enseignement Supérieur
Directeur du Laboratoire Informatique, Intelligence
Artificielle et Cyber Sécurité - ENSET - UH2



Judi 13 Novembre 2025 à 10H



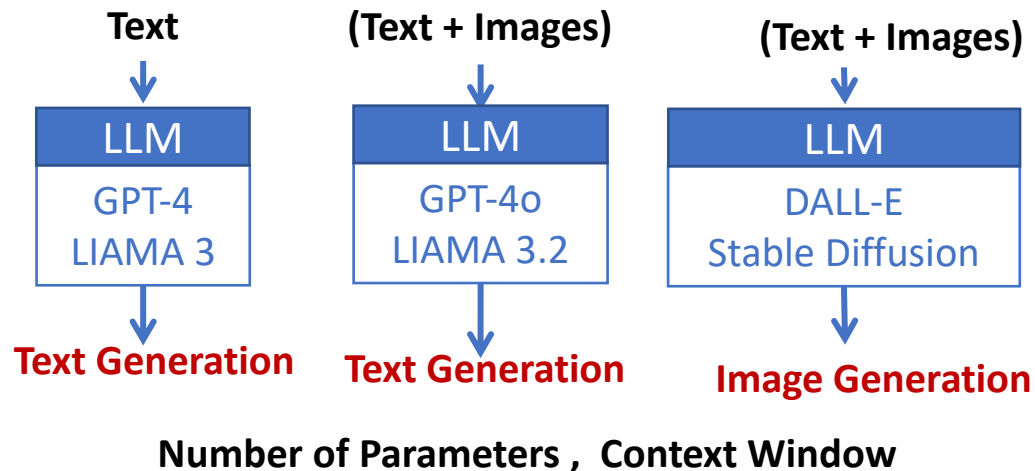
Grand Amphi NB - FSTS



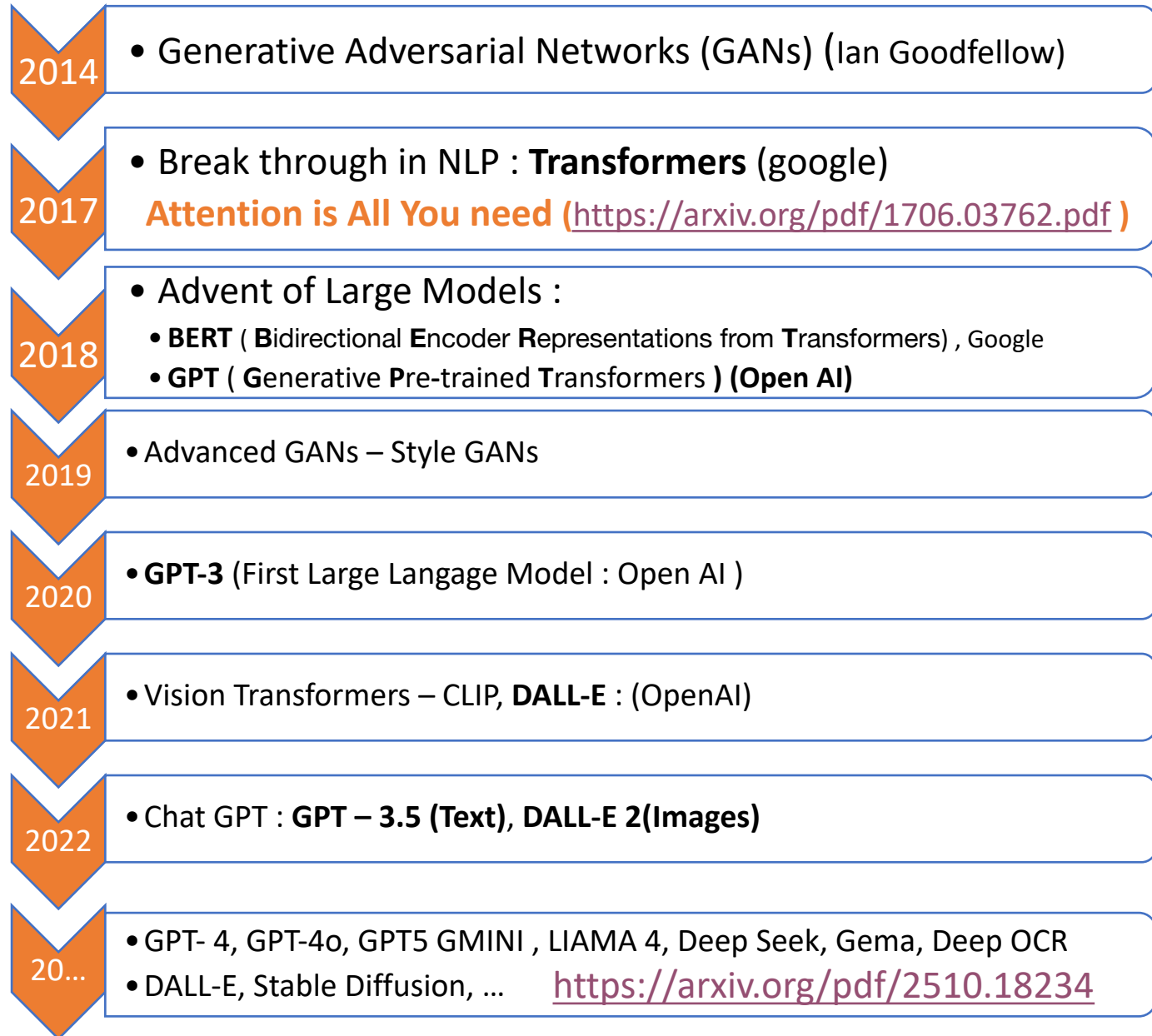
$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Generative AI (State Of Art)

- Generative AI refers to a type of Artificial Intelligence that is capable of generating new data, such as images, music, or text, based on a set of training data.
- Golden age of AI :
 - Powerful of Language Models (LLMs)
 - Abundant Data
 - HCP : GPU & TPU , Cloud Computing



<https://thispersondoesnotexist.com/>

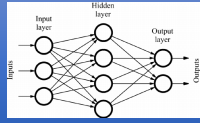


Generative AI (State Of Art)

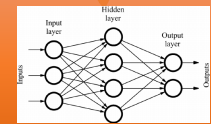
Random Noise seed

0.3	-0.6		0.55	-0.9
-----	------	------	------	------

Generative
Network Model



Discriminator
Network Model



Real or fake?



Generative
Loss

Discriminator
Loss

Re-Train

<https://thispersondoesnotexist.com/>

2014

- Generative Adversarial Networks (GANs) (Ian Goodfellow)

2017

- Break through in NLP : **Transformers** (google)
Attention is All You need (<https://arxiv.org/pdf/1706.03762.pdf>)

2018

- Advent of Large Models :
 - **BERT** (**B**idirectional **E**ncoder **R**epresentations from **T**ransformers) , Google
 - **GPT** (**G**enerative **P**re-trained **T**ransformers) (Open AI)

2019

- Advanced GANs – Style GANs

2020

- **GPT-3** (First Large Language Model : Open AI)

2021

- Vision Transformers – CLIP, **DALL-E** : (OpenAI)

2022

- Chat GPT : **GPT – 3.5** (Text), **DALL-E 2**(Images)

20...

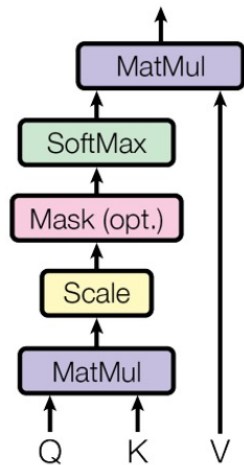
- GPT- 4, GPT-4o, GPT5 GMINI , LIAMA 4, Deep Seek, Gema, Deep OCR
- DALL-E, Stable Diffusion, ... <https://arxiv.org/pdf/2510.18234>

Generative AI (State Of Art)

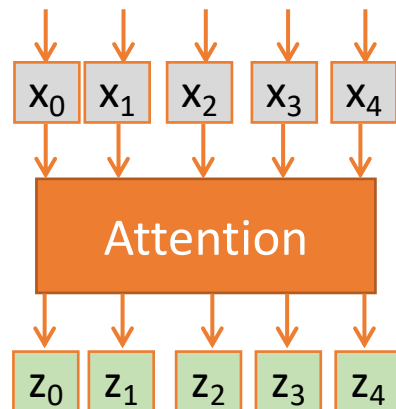
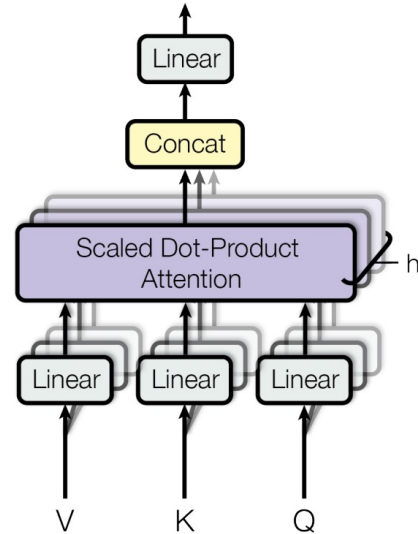
Attention Is All You Need

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Scaled Dot-Product Attention



Multi-Head Attention



<https://thispersondoesnotexist.com/>

2014

- Generative Adversarial Networks (GANs) (Ian Goodfellow)

2017

- Break through in NLP : **Transformers** (google)
Attention is All You need (<https://arxiv.org/pdf/1706.03762.pdf>)

2018

- Advent of Large Models :
 - **BERT** (**B**idirectional **E**ncoder **R**epresentations from **T**ransformers) , Google
 - **GPT** (**G**enerative **P**re-trained **T**ransformers) (**O**pen **A**I)

2019

- Advanced GANs – Style GANs

2020

- **GPT-3** (First Large Language Model : Open AI)

2021

- Vision Transformers – CLIP, **DALL-E** : (OpenAI)

2022

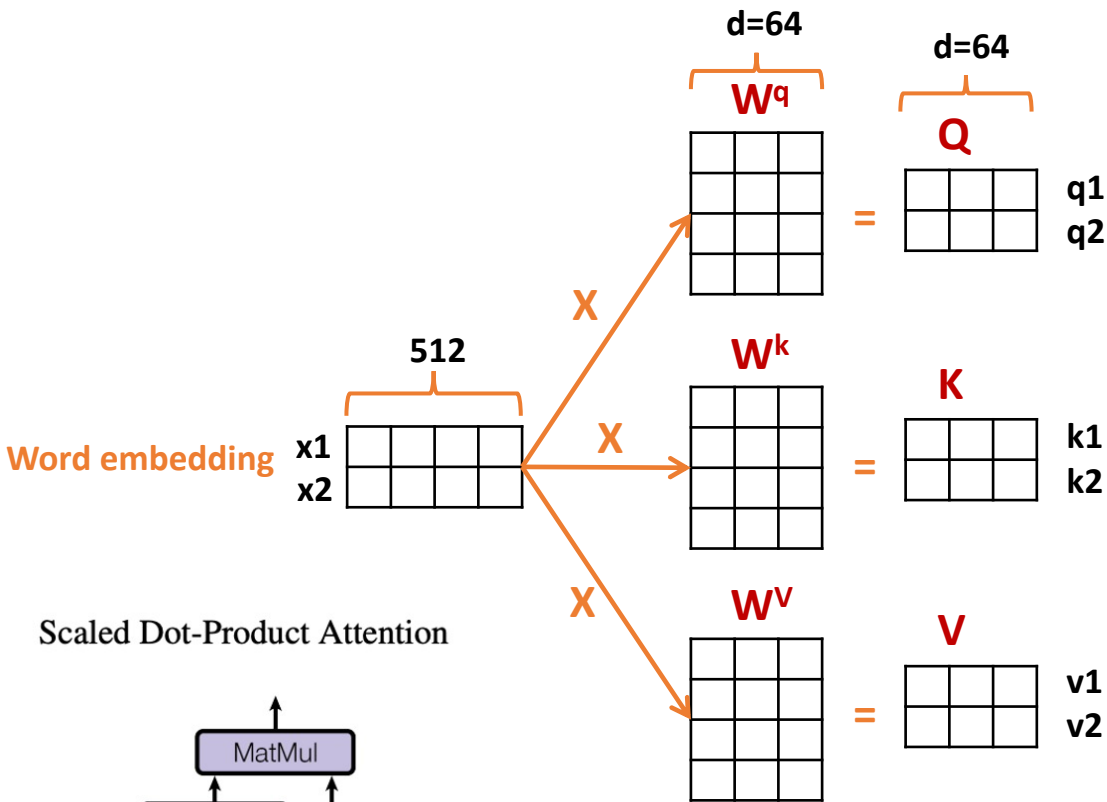
- Chat GPT : **GPT – 3.5 (Text)**, **DALL-E 2(Images)**

20...

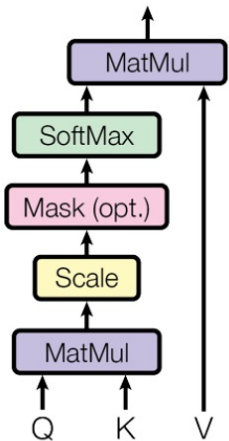
- GPT- 4, GPT-4o, GPT5 GMINI , LIAMA 4, Deep Seek, Gema, Deep OCR
- DALL-E, Stable Diffusion, ... <https://arxiv.org/pdf/2510.18234>

Attention Is All You Need

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



Scaled Dot-Product Attention



Dot Product

Score = $Q \cdot K =$

q1.k1	q1.k2
q2.k1	q2.k2

Score is divided by $\sqrt{d_k}$ to produce the DP matrix:

DP11	DP12
DP21	DP22

SM = SoftMax (DP matrix) =

0.88	0.12
0.11	0.89

Z = SM . V =

0.88	0.12
0.11	0.89

X

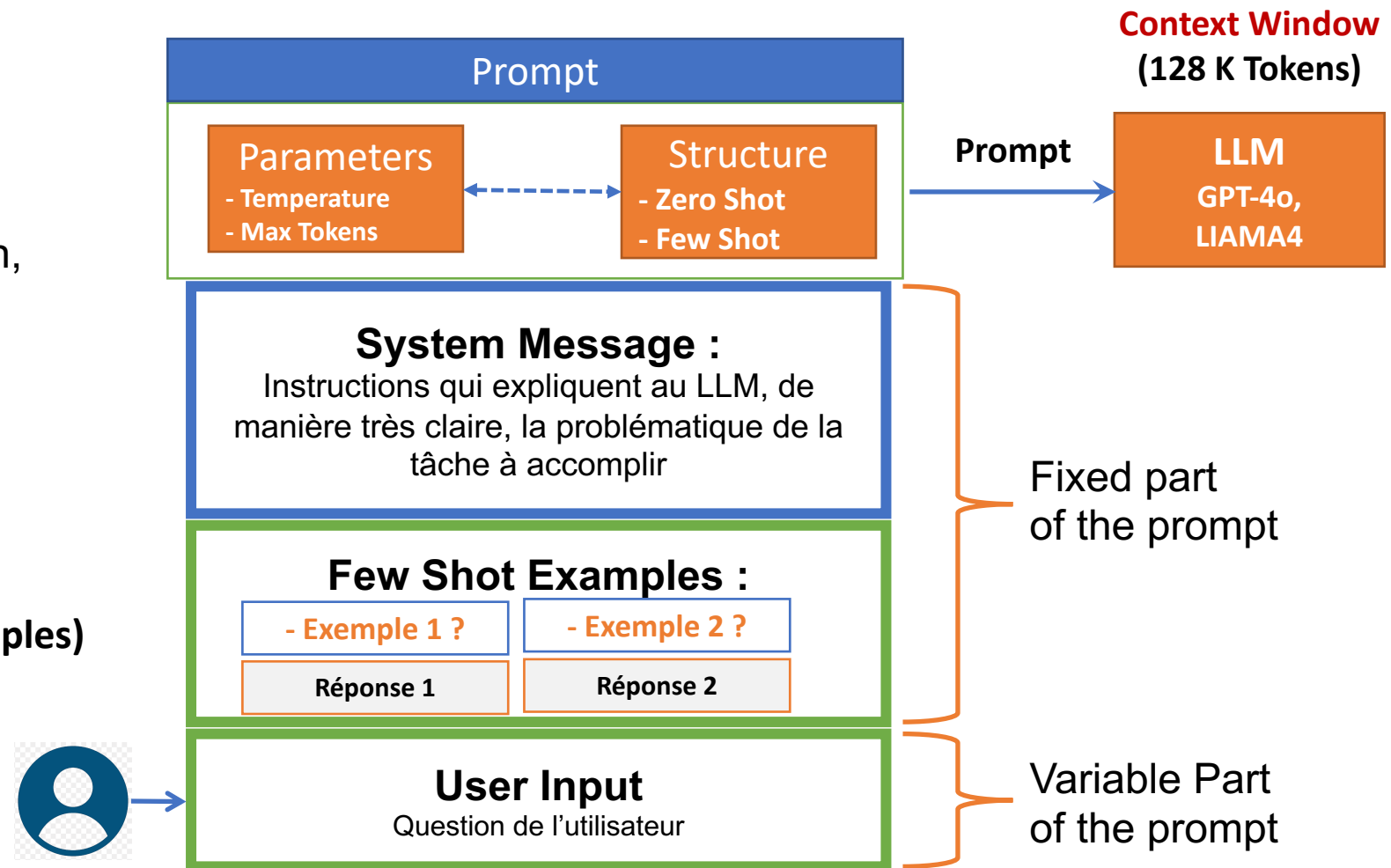
=

0.88	0.12
0.11	0.89

Prompt Engineering

- **Prompt** = Ensemble d'instructions spécifiques à envoyer un LLM pour accomplir une tâche spécifique
- **Engineering** = Processus de conception, création, évaluation et déploiement d'un prompt pour des tâches spécifiques
- **Structure d'un prompt :**
 - Zero Shot Prompt (Sans exemples)
 - Few Shot Prompt (Guidé par des exemples)
 - Chain-of-Thought Prompt (*Invite de la chaine de pensée*)

Model	Context Window
Gpt-3.5-turbo	4096
Gpt-4-tubo	128 000
Mixtral-8x7B	128 000



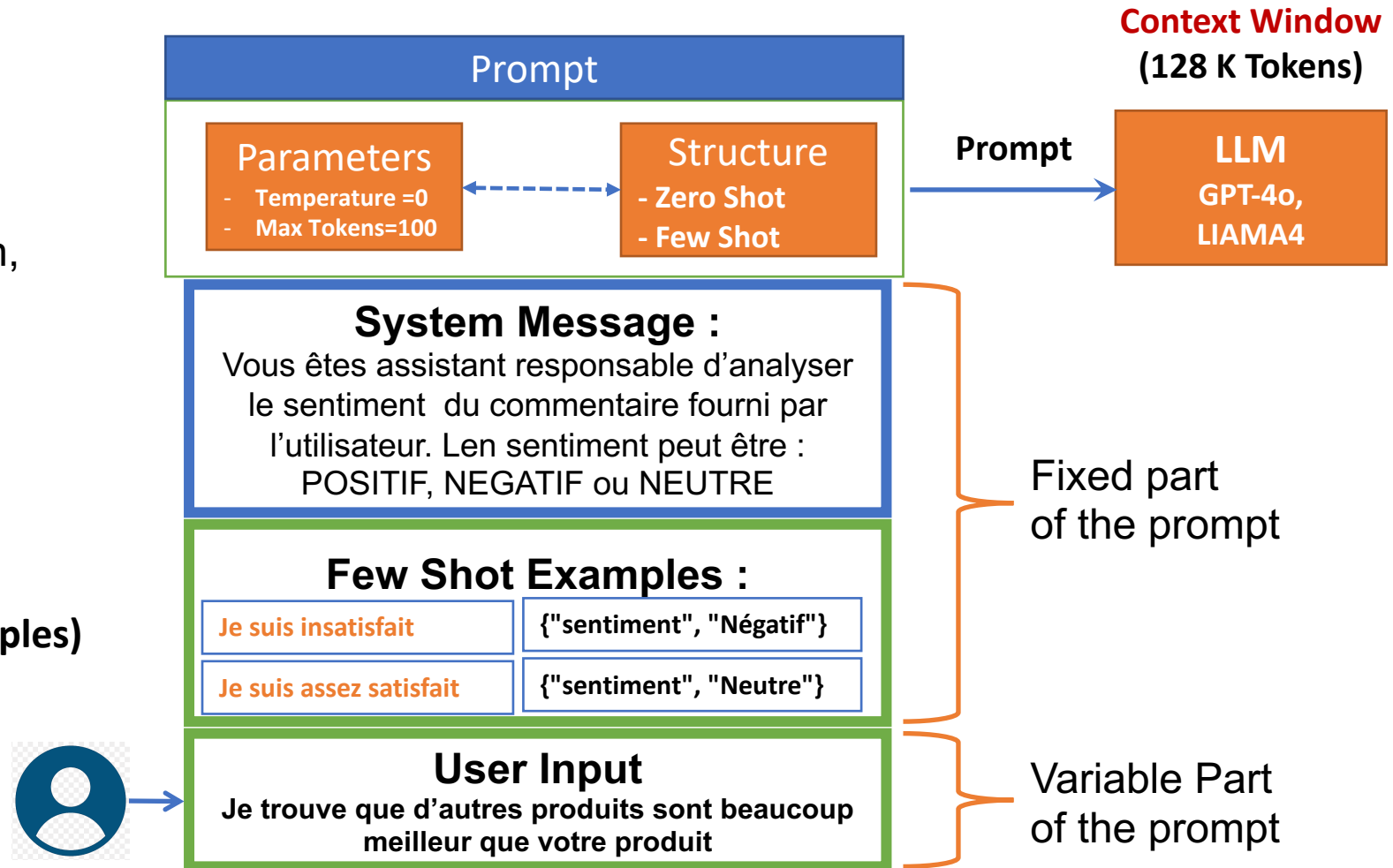
<https://platform.openai.com/tokenizer>

<https://platform.openai.com/>

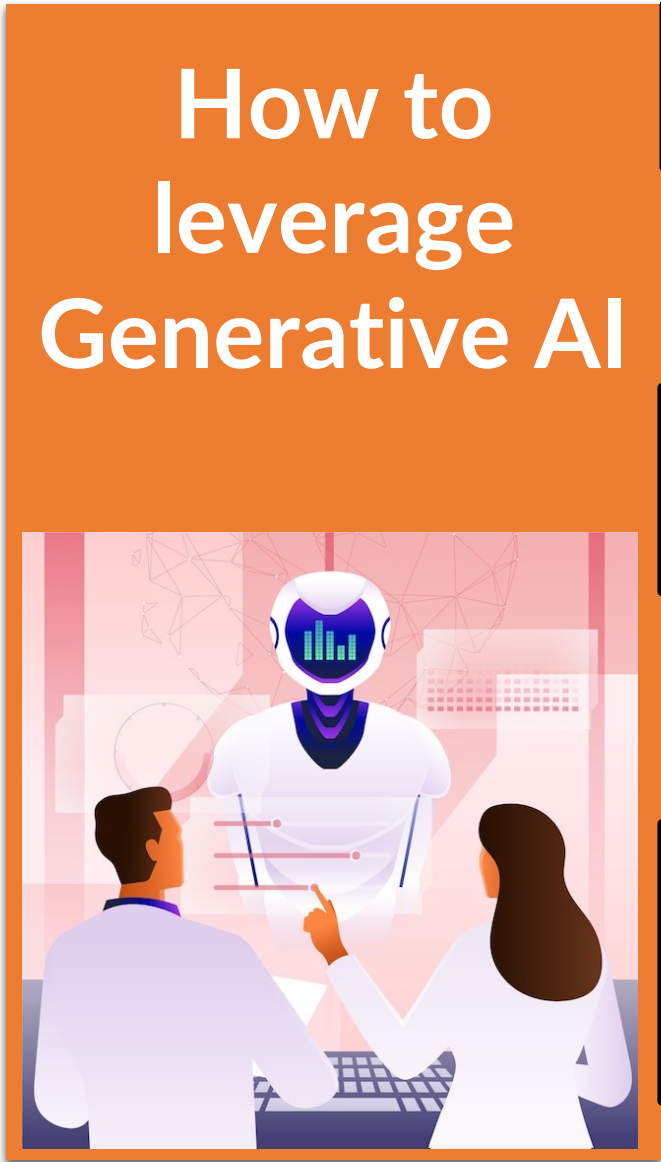
Prompt Engineering

- **Prompt** = Ensemble d'instructions spécifiques à envoyer un LLM pour accomplir une tâche spécifique
- **Engineering** = Processus de conception, création, évaluation et déploiement d'un prompt pour des tâches spécifiques
- **Structure d'un prompt :**
 - Zero Shot Prompt (Sans exemples)
 - Few Shot Prompt (Guidé par des exemples)
 - Chain-of-Thought Prompt (*Invite de la chaine de pensée*)

Model	Context Window
Gpt-3.5-turbo	4096
Gpt-4-tubo	128 000
Mixtral-8x7B	128 000

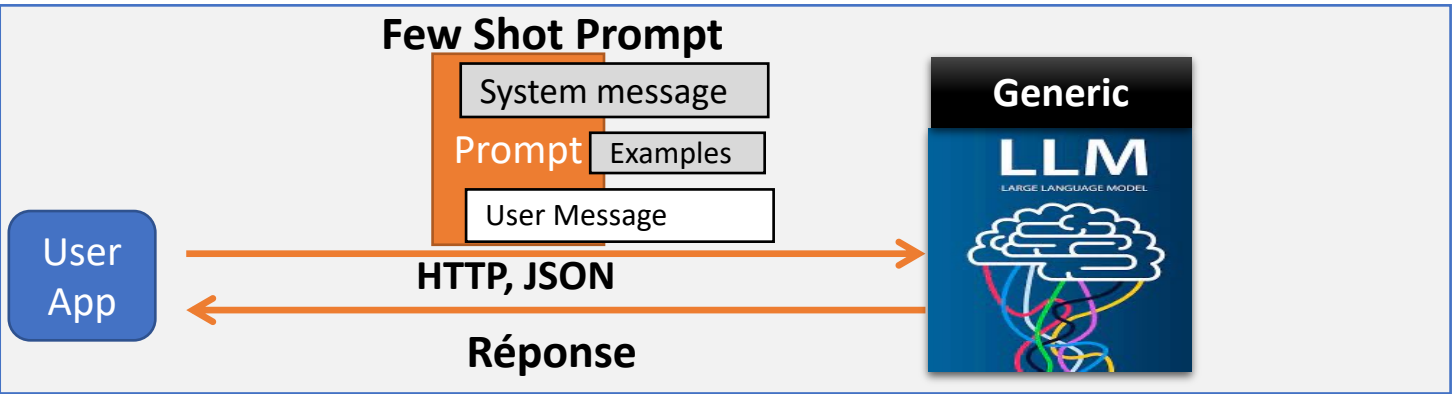


Leverage Generative AI

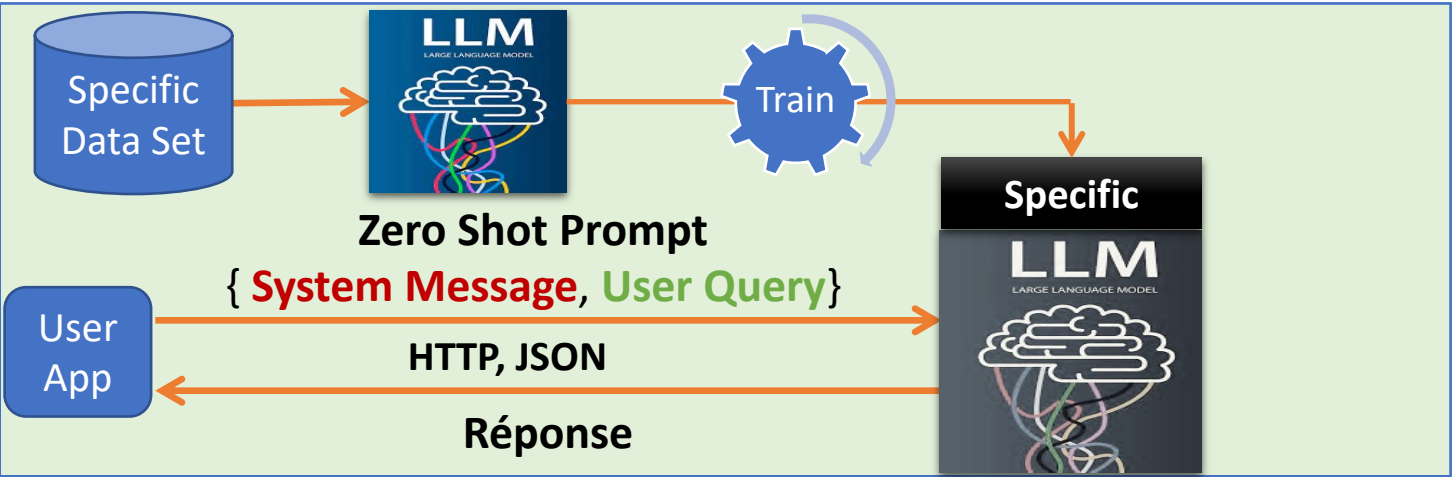


How to leverage Generative AI

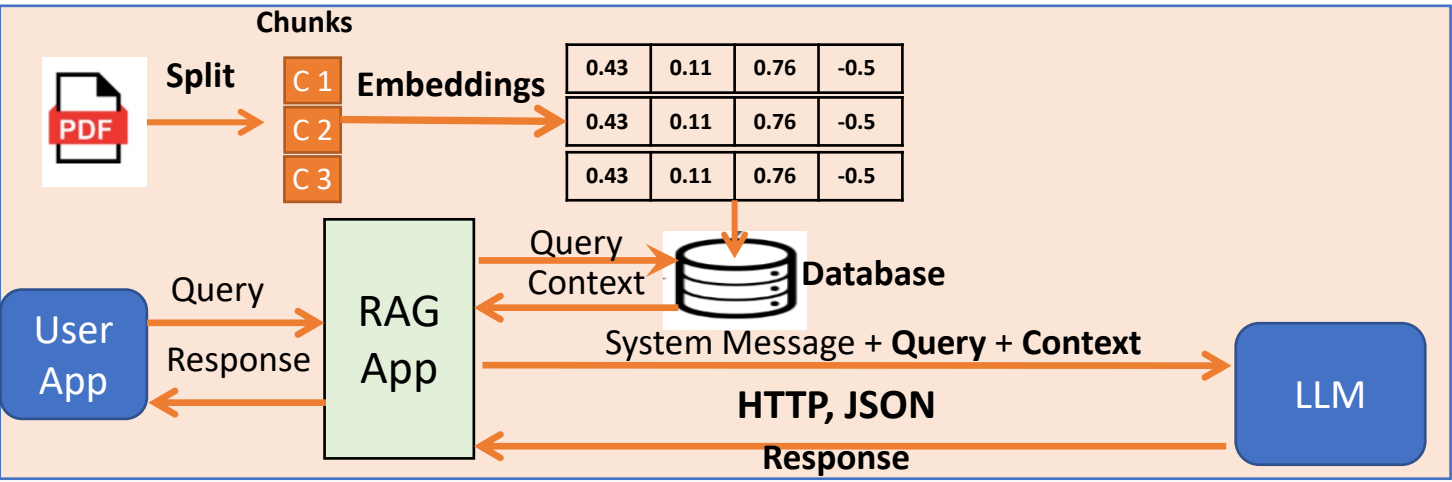
Few Shot Learning



Fine Tuning



RAG
Retrieval
Augmented
Generation

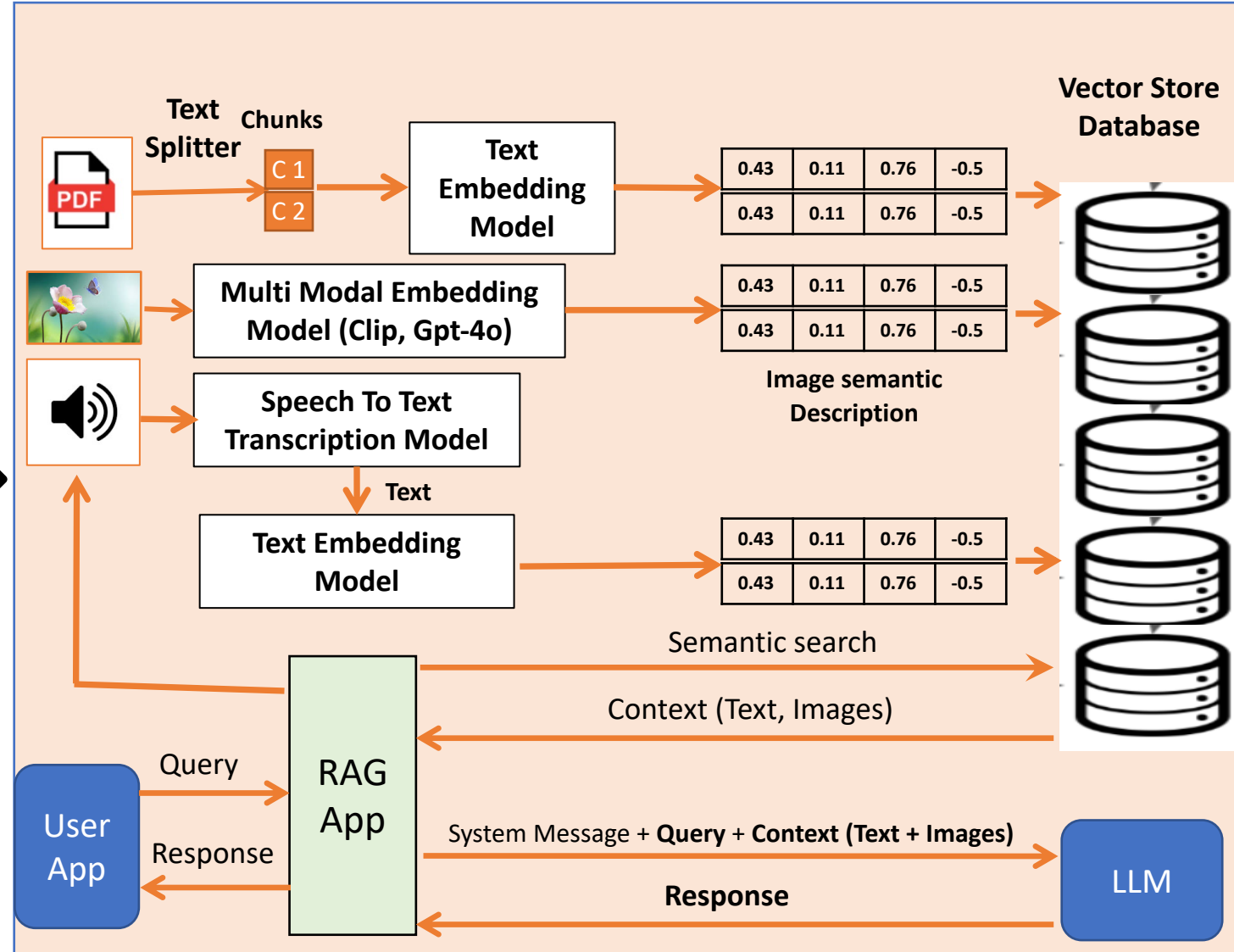


Multi Modal Retrieval Augmented Generation

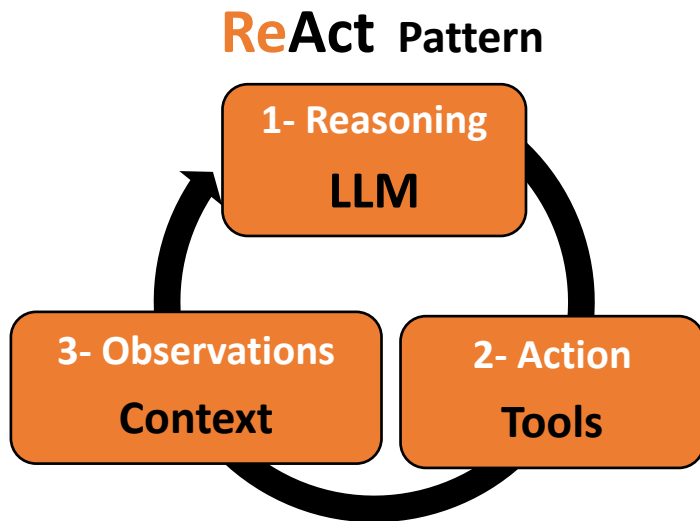
Multi Modal RAG



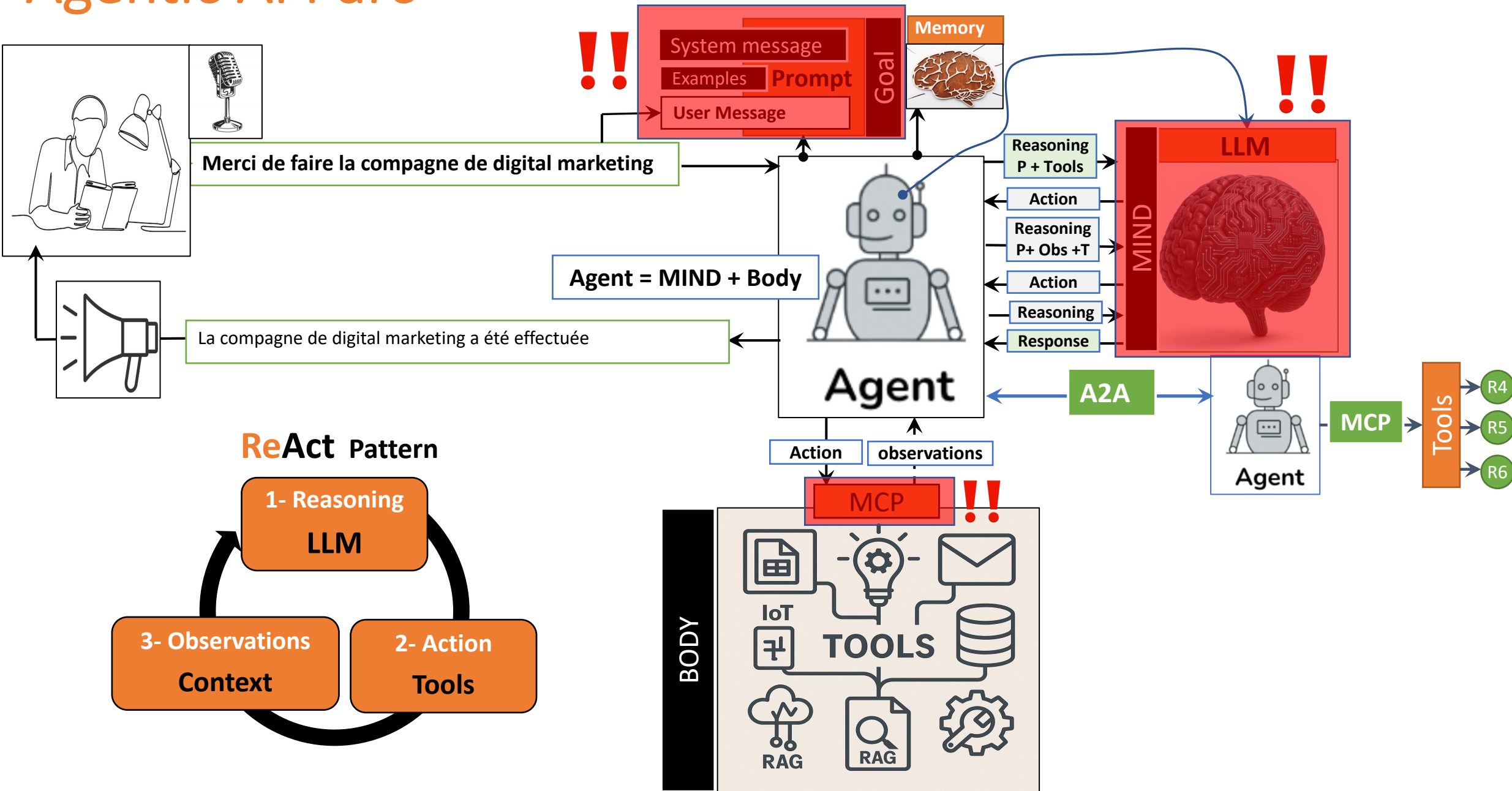
Multi
RAG
Retrieval
Augmented
Generation



The diagram illustrates the ReAct pattern and the Agent architecture. The top section shows a user interaction loop: a user provides a message "Merci de faire la compagne de digital marketing", which is processed by an Agent (MIND + Body) to produce a response "La compagne de digital marketing a été effectuée". The Agent's internal structure includes a "MIND" (LLM) and a "BODY" (Tools). The "MIND" interacts with the "LLM" through a series of steps: Reasoning (P + Tools), Action, Reasoning (P + Obs + T), Action, Reasoning, and Response. The "BODY" contains various tools like IoT, RAG, and a database. The bottom section shows the "ReAct Pattern" as a cycle: 1- Reasoning LLM, 2- Action Tools, and 3- Observations Context. The entire system is connected to a "Goal" (brain icon) and a "System message" (Examples Prompt).

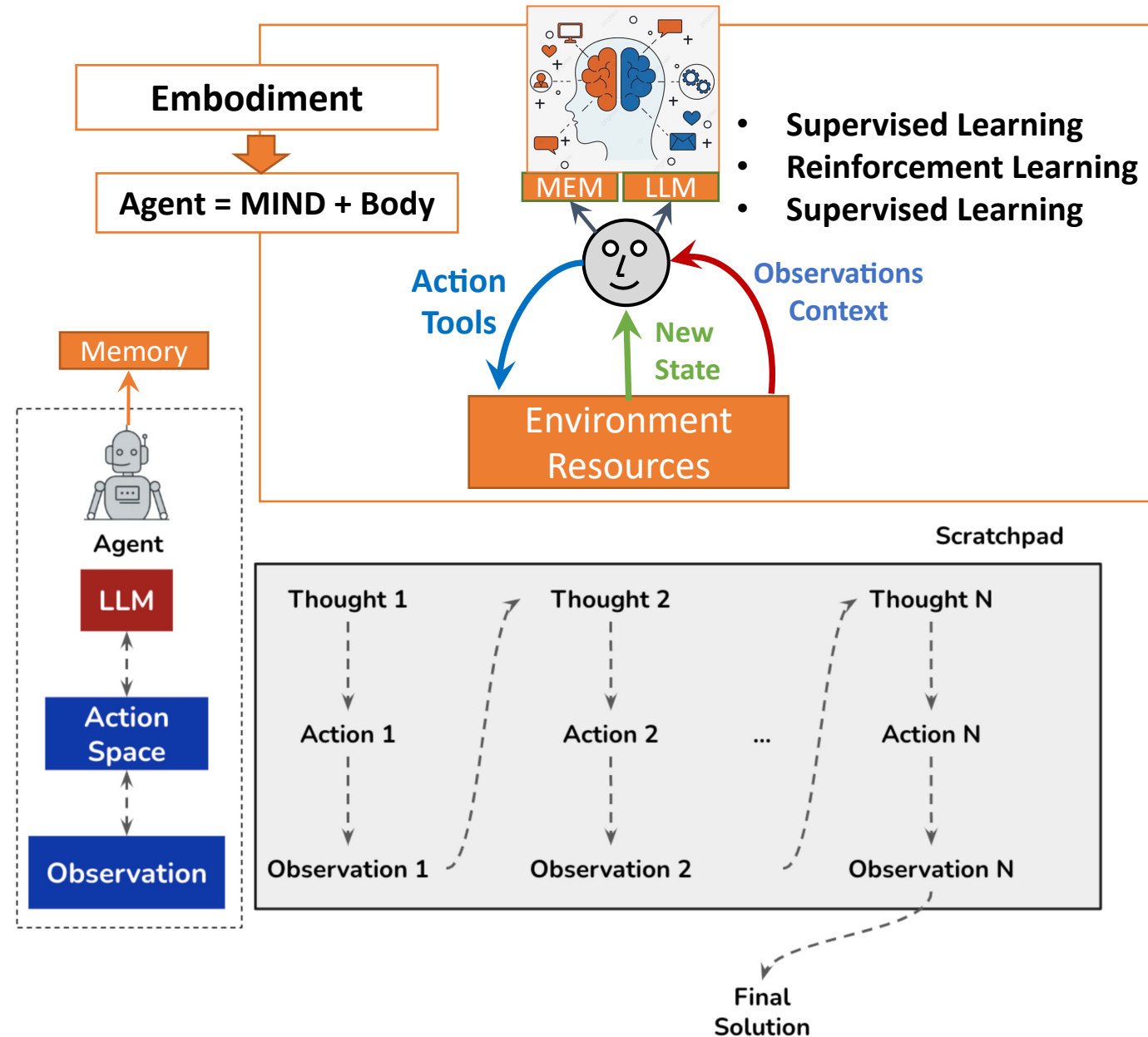
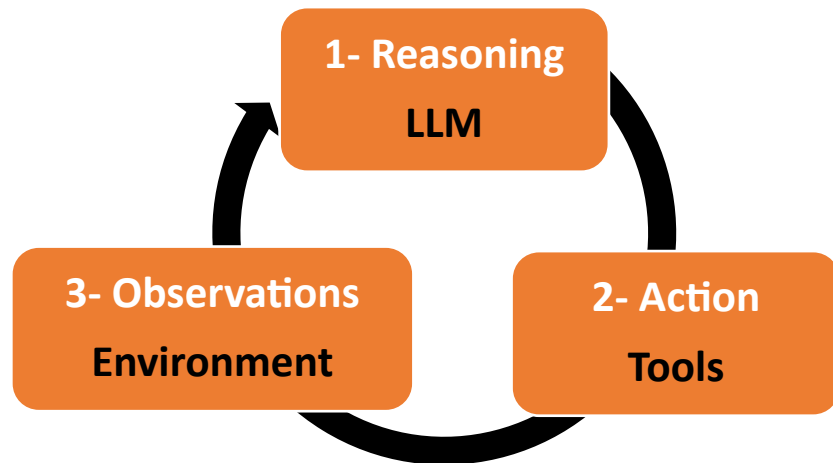


Agentic AI Parc

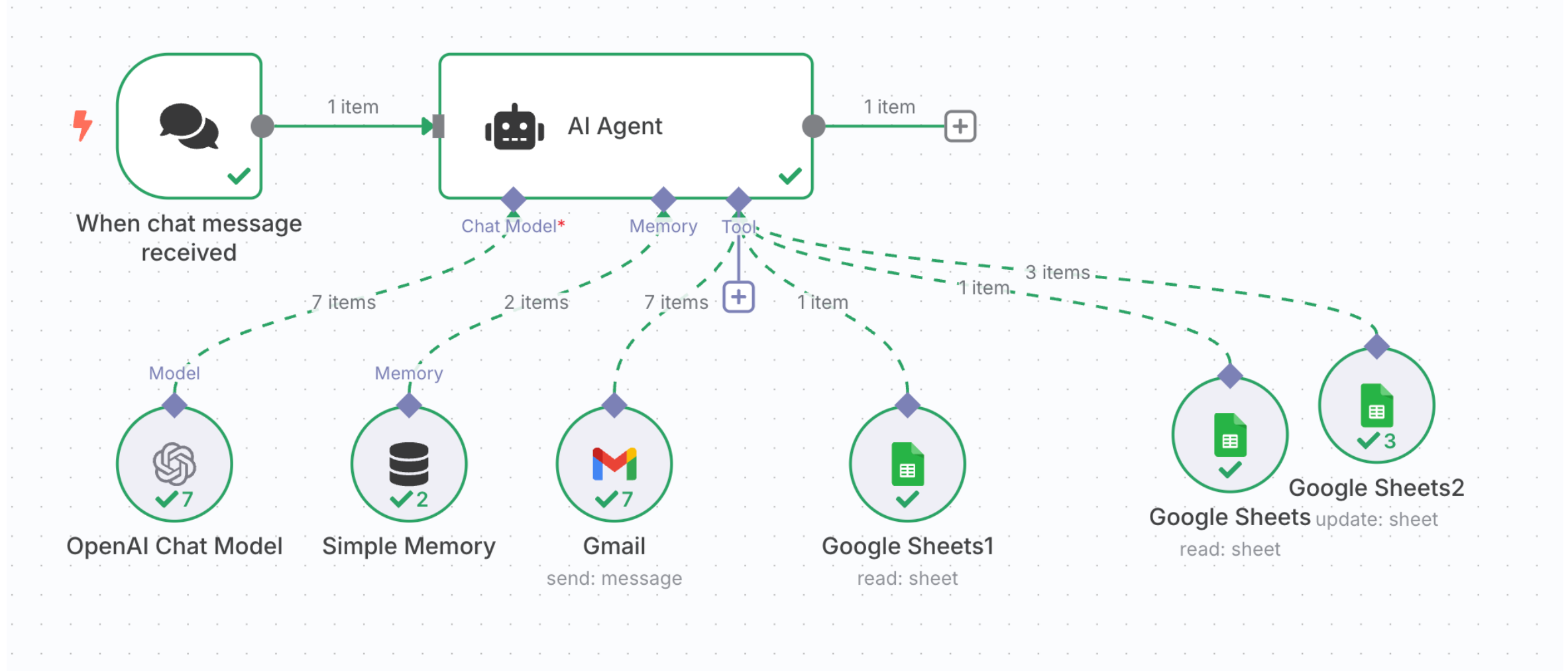


AI Agents

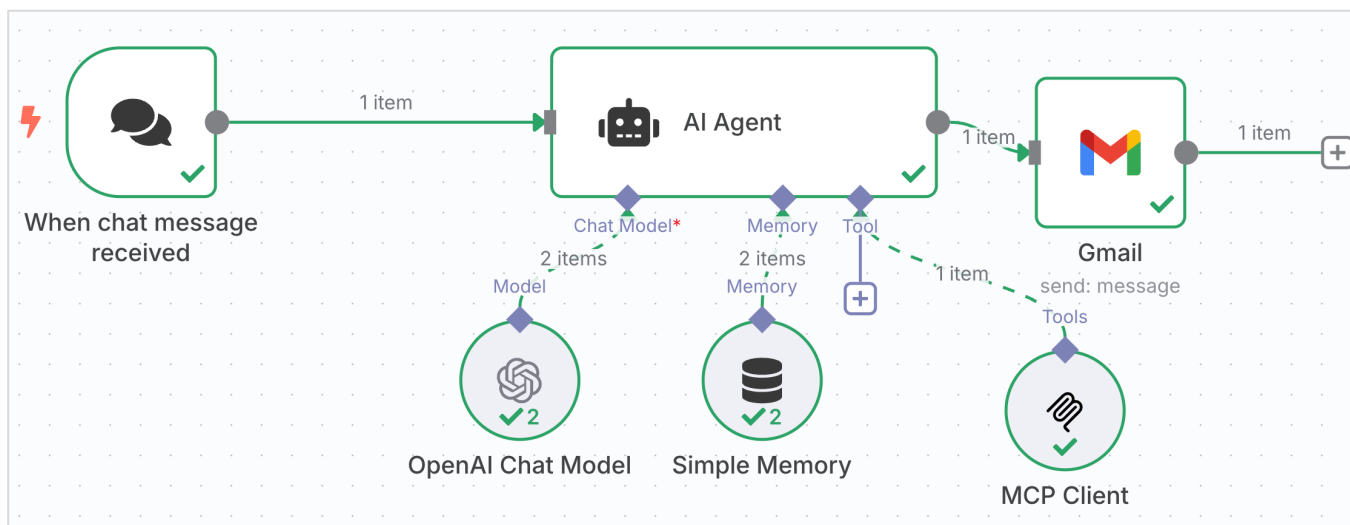
- An AI Agent, as described by the **ReaAct** Framework, is an **autonomous** entity with a goal that can:
 - **Reason** based on context (environmental abstraction)
 - **Act autonomously** on its environment
 - **Collect observations** about the environment



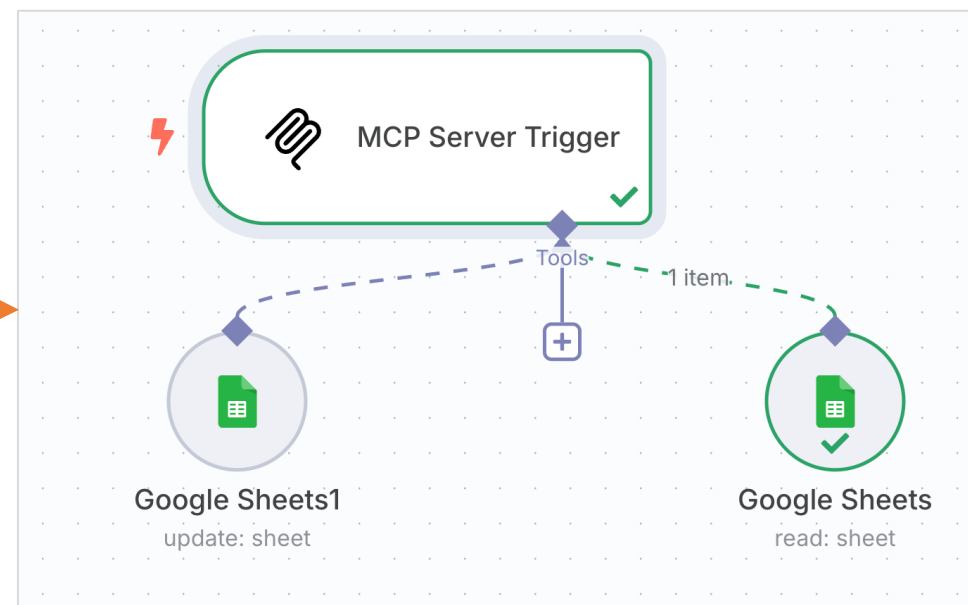
Demo 1 : First Agent in Automation platform : n8n



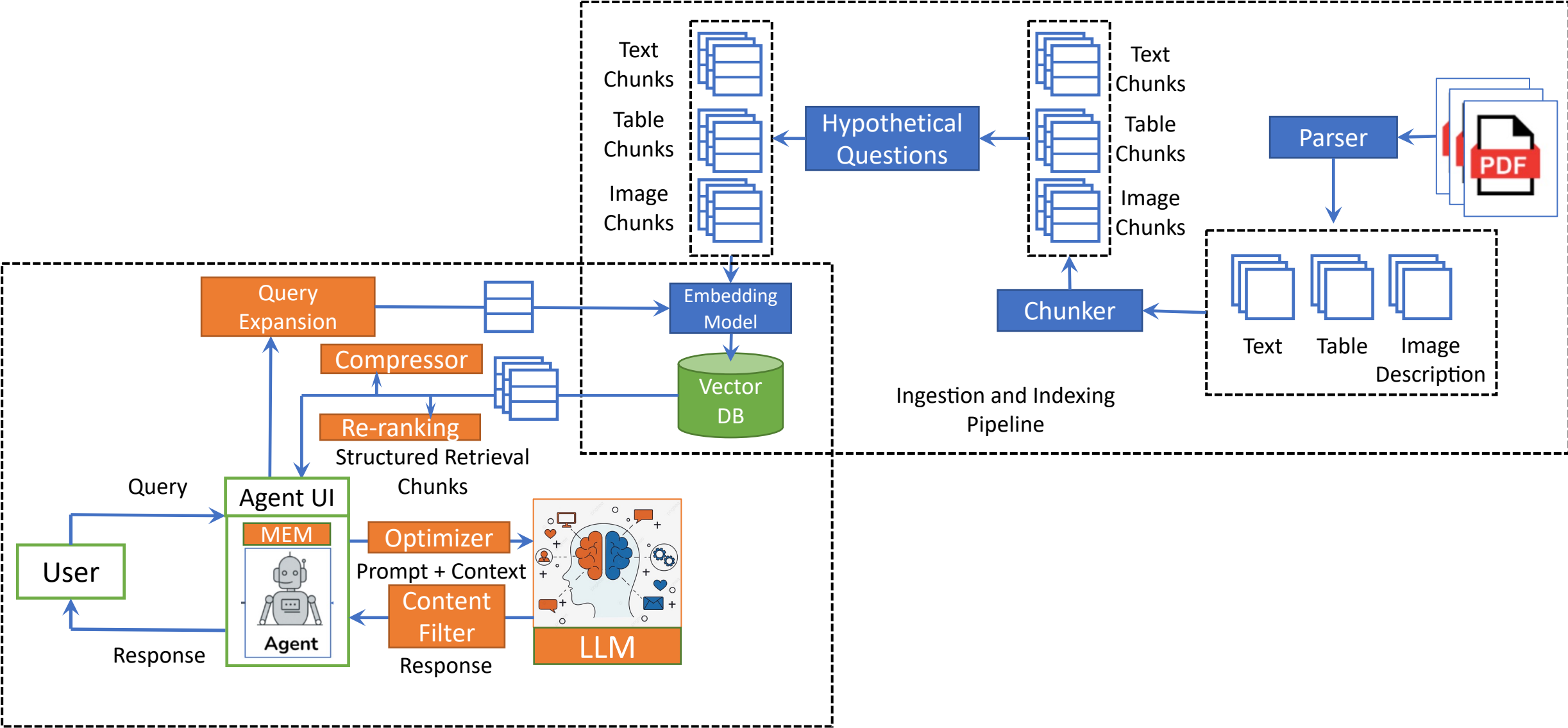
Demo 2 : AI Agent with MCP



MCP



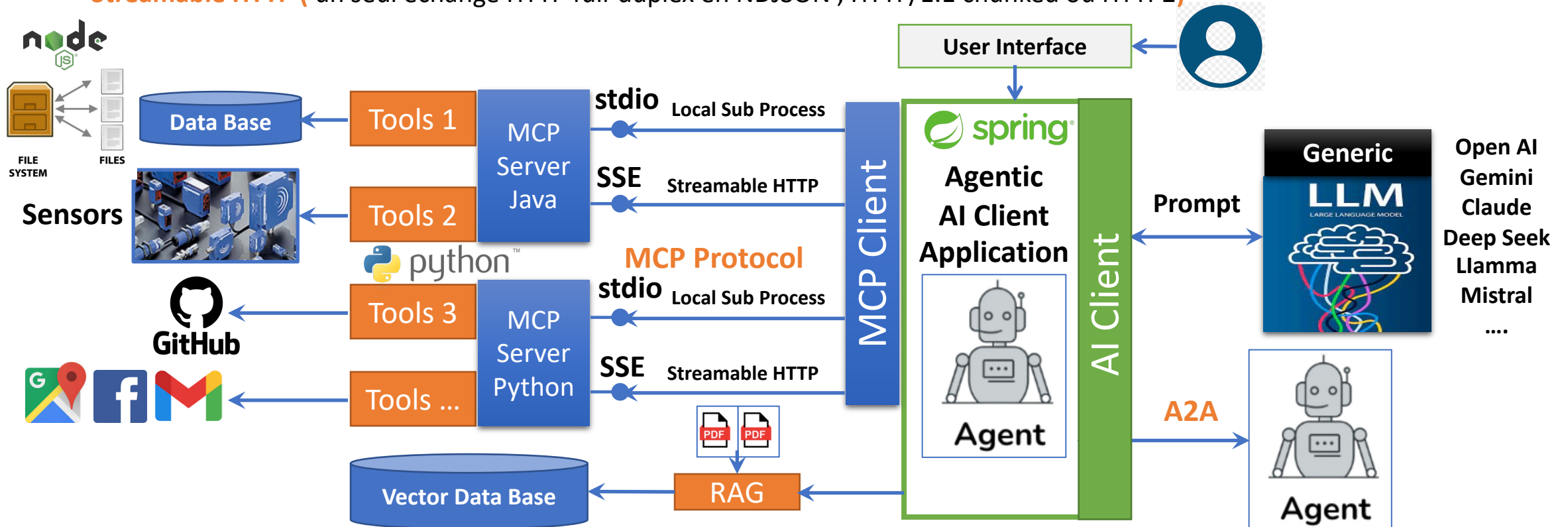
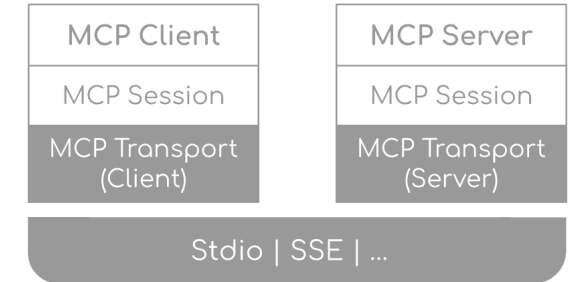
Advanced Multi Modal Agentic RAG



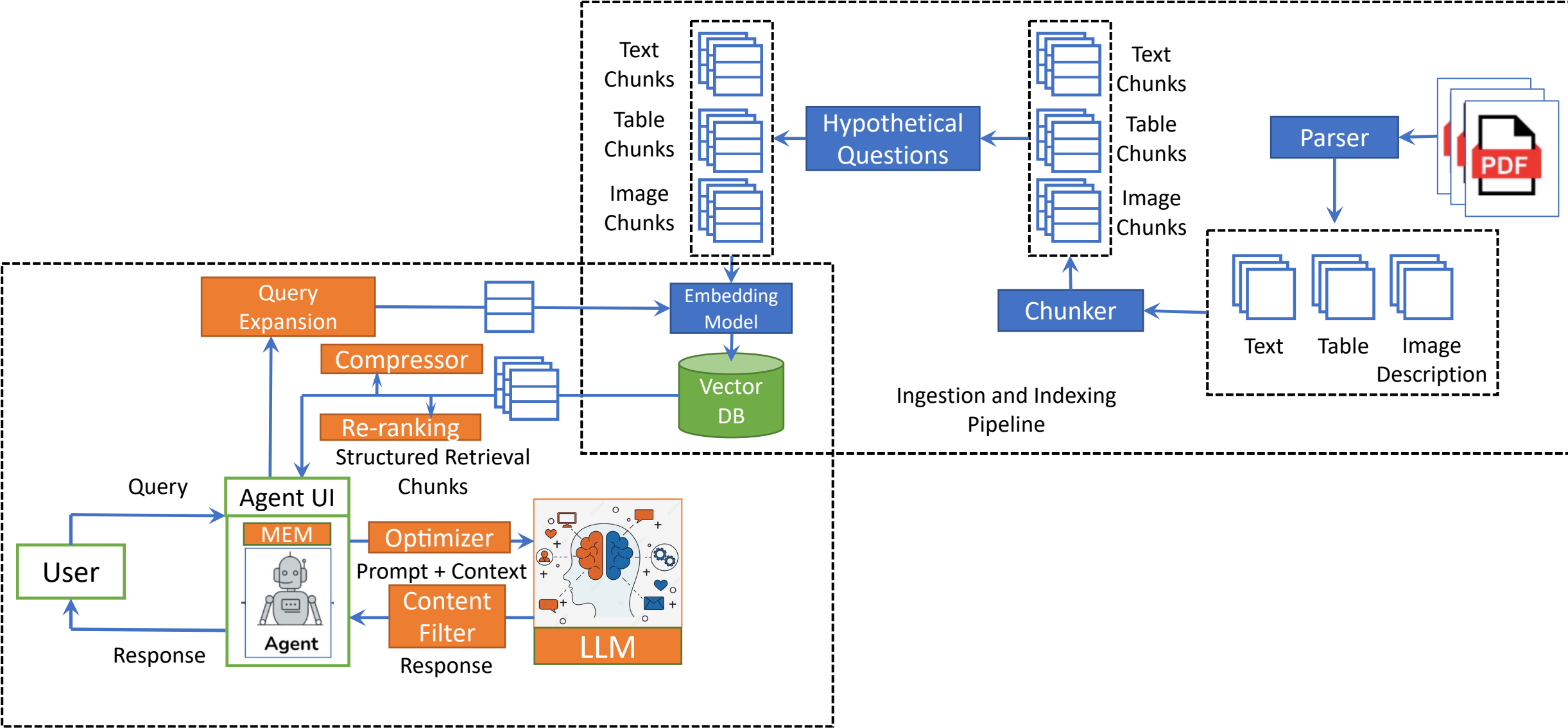
Model Context Protocol (MCP)

<https://modelcontextprotocol.io>

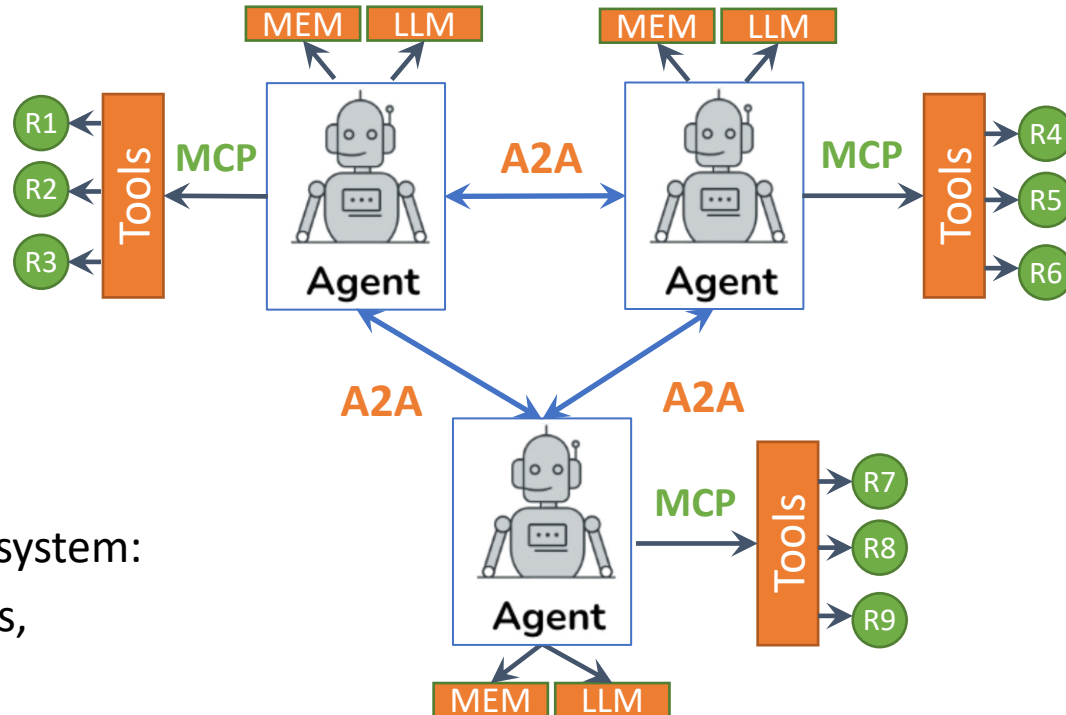
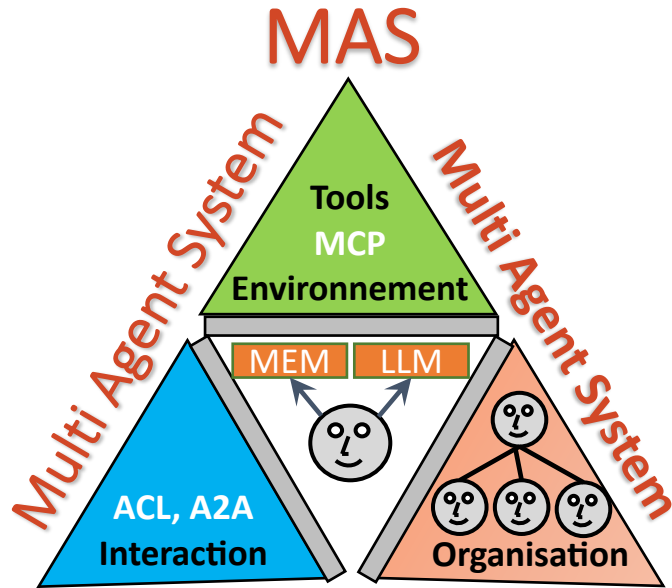
- MCP standardizes how to integrate **additional context and tools** into the ecosystem of AI applications
- The protocol uses **JSON-RPC 2.0 messages**
- The protocol defines two standard transport mechanisms for client-server communication:
 - **STDIO**, communication over **standard input** and **standard output**
 - **SSE (Server Sent Event)** : serveur → client en streaming + POST séparés pour client → serveur
 - **Streamable HTTP** (un seul échange HTTP full-duplex en NDJSON , HTTP/1.1 chunked ou HTTP2)



Advanced Multi Modal Agentic RAG

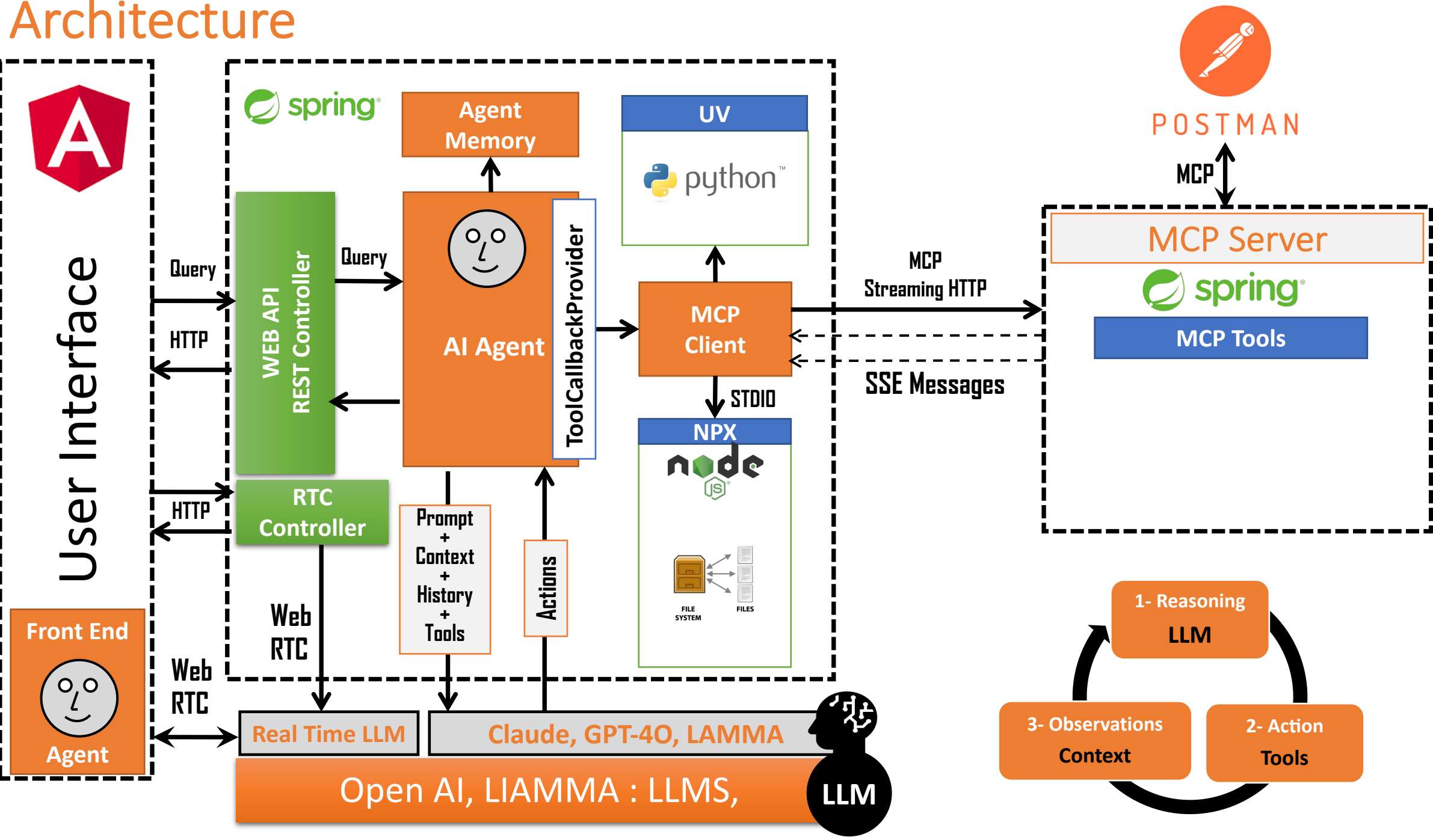


MAS And new Protocols : MCP et A2A



- A multi-agent system (MAS) is a distributed system:
 - Composed of a set of distributed agents,
 - Located in a specific environment
 - and Interacting according to specific organizational structures.
- An MAS solves complex problems by leveraging the collective intelligence of its component agents (DAI).

Architecture



AI Agents Frameworks

- **OpenAI Agent SDK** : <https://openai.github.io/openai-agents-python/>
- **CrewAI** : <https://docs.crewai.com/introduction>
- **LangChain** : <https://python.langchain.com/docs/tutorials/agents/>
- **AutoGen** : <https://microsoft.github.io/autogen/dev//user-guide/agentchat-user-guide/tutorial/agents.html>
- **Pydantic AI Agent** : <https://ai.pydantic.dev/>
- **Phidata** : <https://docs.phidata.com/introduction>
- **PraisonAiAgents** : <https://docs.praison.ai/framework/praisonaigents>
- ...

OpenAI Agents SDK

- The OpenAI Agents SDK aims to build agentic AI apps in a lightweight, easy-to-use package with very few abstractions.
- It's a production-ready upgrade of our previous experimentation for agents, Swarm.
- The Agents SDK has a very small set of primitives:
 - **Agents**, which are LLMs equipped with instructions and tools
 - **Handoffs**, which allow agents to delegate to other agents for specific tasks
 - **Guardrails**, which enable the inputs to agents to be validated
 - Support **Model Context Protocol (MCP)**

Installation

```
pip install openai-agents
```

Hello world example

```
from agents import Agent, Runner

agent = Agent(name="Assistant", instructions="You are a helpful assistant")

result = Runner.run_sync(agent, "Write a haiku about recursion in programming.")
print(result.final_output)

# Code within the code,
# Functions calling themselves,
# Infinite loop's dance.
```

```
export OPENAI_API_KEY=sk-...
```

DEMO : Agentic AI

Source code and resources : <https://github.com/mohamedYoussfi/agentic-ai>

```
> pip install uv
> uv init
> uv venv
> source .venv/bin/activate (MACOS)
> .venv\Scripts\activate.bat (Windows)
> uv add ipykernel
> uv add "openai-agents[viz]"
> uv add pydantic
> uv add python-dotenv
> uv add streamlit
```

• pyproject.toml

```
[project]
name = "test"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "duckduckgo-search>=8.0.1",
    "ipykernel>=6.29.5",
    "openai-agents[viz]>=0.0.14",
    "pydantic>=2.11.4",
    "python-dotenv>=1.1.0",
    "streamlit>=1.45.0",
]
```

First Agent Using OpenAI Model

```
from dotenv import load_dotenv
import os
from IPython.display import Markdown
```

✓ 0.0s

```
load_dotenv()
```

✓ 0.0s

```
api_key = os.environ.get('OPENAI_API_KEY')
if not api_key:
    raise("API KEY not available")
```

✓ 0.0s



```
OPENAI_API_KEY=sk-proj-...
```

```
from agents import Agent, Runner, OpenAIChatCompletionsModel, AsyncOpenAI
```

✓ 0.5s

```
cost_medico_agent = Agent(
    name="Doctor Agent",
    model="gpt-4o",
    instructions="""
    Donne moi des conseils concernant la pathologie fournie par l'utilisateur
    """
)
result = await(Runner.run(cost_medico_agent,"Cardiologie"))
```

```
display(Markdown(result.final_output))
```

En ce qui concerne la cardiologie, il est important de prendre soin de la santé cardiovasculaire en suivant plusieurs conseils généraux:

1. **Alimentation équilibrée** : Adopte un régime riche en fruits, légumes, grains entiers, protéines maigres et acides gras oméga-3 (présents dans le poisson, les noix et les graines).
2. **Activité physique régulière** : Vise au moins 150 minutes d'exercice modéré par semaine, comme la marche rapide, la natation ou le vélo.
3. **Gestion du stress** : Pratiques comme le yoga, la méditation ou des exercices de respiration peuvent aider à réduire le stress.
4. **Arrêt du tabac** : Si tu fumes, chercher de l'aide pour arrêter peut drastiquement améliorer ta santé cardiaque.
5. **Contrôle de la pression artérielle et du cholestérol** : Effectue des contrôles réguliers et prends les mesures nécessaires si les niveaux sont élevés.
6. **Maintien d'un poids sain** : Un poids adéquat peut réduire le risque de maladies cardiovasculaires.
7. **Modération de la consommation d'alcool** : Limite ta consommation d'alcool si tu choisis de boire.
8. **Suivi médical** : Effectue des bilans de santé réguliers pour détecter précocement les problèmes.

Si tu as des soucis spécifiques ou un diagnostic, il est important de consulter un cardiologue pour un plan de traitement adapté à ton cas.

First Agent Using OpenAI Llama 3.2 Model

```
llama_model = OpenAIChatCompletionsModel(  
    ... model="llama3.2",  
    ... openai_client= AsyncOpenAI(base_url="http://localhost:11434/v1")  
)
```

```
free_medico_agent = Agent(  
    name="Doctor Agent",  
    model=llama_model,  
    instructions="""  
    Donne moi des conseils concernant la pathologie fournie par l'utilisateur  
    """"  
)  
  
result = await(Runner.run(free_medico_agent,"Cardiologie"))
```

```
display(Markdown(result.final_output))
```

Voici quelques informations générales et conseils relatifs à la cardiologie :

Qu'est-ce que le cœur ?

Le cœur est un organe essentiel de notre corps qui frappe sur environ 100 000 fois par jour. Il est responsable de pousser l'oxygène à travers les artères et d'amener à retourner au cœur le déchet, comme les déchets du cerveau. Le cœur est établi à la base du torso.

Les troubles cardiaques

Il existe une large gamme de maladies cardiovasculaires qui affectent millions de personnes dans le monde entier. Voici quelques exemples :

1. **Insuffisance cardiaque**: Lorsqu'elles ne fonctionnent pas avec suffisamment pour fournir les doses nécessaires à tout le corps, les organes peuvent s'empirer.
2. **Coronaropathie** : Cette maladie est due au stress des artères. Tout ce que vous avez besoin de faire, c'est que votre système d'ongulats réagisse très bien avec une pression.
3. **Aortique et pulmonaire hypertension**: le risque de stent ou d'assister à un cedran
4. **Arérodite hémorragique** : sache qu'il y a peut-être des problèmes en cas d'hémorragie
5. **Anomalies du cœur (AESC)**

```
mohamedyoussefi — fina@aifinashore: ~ — ollama run llama3.2 — 74x16  
Last login: Sun May 11 08:29:31 on ttys027  
(base) $ ollama ls  
NAME                                ID                                SIZE    MODIFIED  
llama3.2:latest                     a80c4f17acd5                     2.0 GB  2 days ago  
deepseek-r1:7b                      0a8c26691023                     4.7 GB  3 months ago  
mxbai-embed-large:latest            468836162de7                     669 MB  5 months ago  
llama3.2-vision:latest              38107a0cd119                     7.9 GB  6 months ago  
llama3.1:latest                     62757c860e01                     4.7 GB  9 months ago  
llama2:latest                       78e26419b446                     3.8 GB  10 months ago  
llava:latest                        8dd30f6b0cb1                     4.7 GB  11 months ago  
llama3:latest                       71a106a91016                     4.7 GB  12 months ago  
mistral:latest                      61e88e884507                     4.1 GB  13 months ago  
(base) $ ollama run llama3.2  
>>> end a message (/? for help)
```

Agent and Structured Output

```
from pydantic import BaseModel
```

```
class Recipe(BaseModel):  
    title : str  
    ingredients : list[str]  
    cooking_time : int  
    servings : int
```

```
recipe_agent = Agent(  
    name = "Recipe Agent",  
    model="gpt-4o",  
    instructions= """  
You are an agent for creating Recipes  
You will be given the name of the food to create  
You job is to create this food as an actual detailed recipe  
The cooking time should be in minutes  
""",  
    output_type= Recipe  
)
```

```
response = await Runner.run(recipe_agent,"Tajine Marocain en Arabe")  
recipe = response.final_output
```

```
print("*"*40)  
print(f>Title : {recipe.title}")  
print(f>Time : {recipe.cooking_time} minutes")  
print(f>Servings : {recipe.servings}")  
for ing in recipe.ingredients:  
    print(f>- {ing}")
```

Title : طاجين مغربي

Time : 120 minutes

Servings : 4

- كغ من لحم الغنم (أو الدجاج حسب الرغبة) 1
- بصلات متوسطة الحجم، مقطعة إلى شرائح 2
- فصوص ثوم مفرومة 3
- ملاعق كبيرة من زيت الزيتون 4
- ملعقة صغيرة من الزنجبيل المطحون 1
- ملعقة صغيرة من الكركم 1
- ملعقة صغيرة من القرفة 1
- ملح وفلفل حسب الرغبة
- غرام من الزبيب 100
- غرام من اللوز المقلي 100
- عصير نصف ليمونة
- كوب من الماء 2
- حفنة من الكزبرة الطازجة، مفرومة للزينة
- غرام من الخضروات الموسمية (مثل الجزر والبطاطس)، مقطعة 200

Agent and Tools

```
from agents import Agent, function_tool, WebSearchTool
```

```
@function_tool
```

```
def get_weather(city : str) -> str:
```

```
    """
```

```
    Get the weather of a given city
```

```
    """
```

```
    print(f"The tool get_weather is used by the agent to get the weather of the {city}")
```

```
    return f"The weather of {city} is sunny"
```

```
@function_tool
```

```
def get_temperature(city : str) -> str:
```

```
    """
```

```
    Get the temperature in a given city
```

```
    """
```

```
    print(f"The tool get_temperature is used by the agent to get the temperature of the {city}")
```

```
    return f"it is 50° In {city}"
```

```
weather_agent = Agent (
```

```
    name="The Weather Agent",
```

```
    instructions="""
```

```
    As the local weather agent, you will be asked to give information  
    about weather of a given city,
```

```
    Your output should include weather information asked by the user"
```

```
    """
```

```
    tools=[get_weather, get_temperature]
```

```
)
```

```
response = await Runner.run(weather_agent, "The temperature and weather in Marrakech")  
response.final_output
```

The tool get_weather is used by the agent to get the weather of the Marrakech
The tool get_temperature is used by the agent to get the temperature of the Ma

'The weather in Marrakech is sunny, with a temperature of 50°F.'

```
response = await Runner.run(weather_agent, "Quel temps fera t-il demain à Tanger")  
response.final_output
```

The tool get_weather is used by the agent to get the weather of the Tanger

'Demain à Tanger, le temps sera ensoleillé. ☀'

Agent and Tools

```
search_agent = Agent(  
    ... name="Search Agent",  
    ... instructions="""  
    ... You are a news reporter,  
    ... your job is to search new recent articles using internet about a given country  
    ... """,  
    ... tools=[WebSearchTool()]  
)
```

```
from duckduckgo_search import DDGS  
from datetime import datetime
```

```
now = datetime.now().strftime("%Y-%m-%d")  
now
```

```
@function_tool  
def get_news_articles(topic:str):  
    print(f"Running DuckDuckSearch for news in topic : {topic}")  
    ddgs_api = DDGS()  
    results = ddgs_api.text(f"{topic}, {now}", max_results=5)  
    return results
```

```
response = await Runner.run(search_agent, "Les actualités au Maroc")
```

```
display(Markdown(response.final_output))
```

```
search_agent = Agent(  
    name= "Search Agent",  
    instructions= """  
    You are a news reporter,  
    your job is to search new recent articles using internet about a given country  
    """,  
    tools=[get_news_articles]  
)
```

Running DuckDuckSearch for news in topic : المغرب

```
display(Markdown(response.final_output))
```

✓ 0.0s

آخر الأخبار حول المغرب

- حكومة بيدرو سانثيز تمنح المغرب 340 مليون يورو لبناء مشروع محطة تحلية المياه.
الرابطة: [اقرأ المزيد](#)
...مقتطف: حكومة بيدرو سانثيز منحت المغرب مبلغ 340 مليون يورو لتمويل مشروع محطة لتحلية المياه
- التقويم والأعياد الرسمية للمغرب سنة 2025.
الرابطة: [اقرأ المزيد](#)
...مقتطف: تشمل العطلة الرسمية في المغرب عام 2025 رأس السنة الجديدة وذكرى بيان الاستقلال
- المملكة المغربية لسنة 2025 - تفاصيل العطل

Agent and Handoffs

```
from IPython.display import Markdown
from agents.extensions.visualization import draw_graph
```

```
from agents import Agent
from pydantic import BaseModel
```

```
class Tutorial(BaseModel):
    outline : str
    tutorial : str
```

```
tutorial_response = await Runner.run(outline_builder_agent,
                                     "Boucles en Java avec des explications en Français")
tuto = tutorial_response.final_output
```

```
tutorial_generator_agent = Agent(
    name="Tutorial Generator Agent",
    handoff_description= "Used to generate tutorial for a given outline",
    instructions="""
        Given a programming topic and an outline,
        your job is to generate code snippets for each section of the outline,
        Format the tutorial in Markdown using a mix of text explanation and code snippets examples",
        Include comments in the code snippets to further explain the code"
    """,
    output_type=Tutorial
)
```

```
display(Markdown("### Outline :"))
display(Markdown(tuto.outline))
print("*"*60)
display(Markdown("### Totorial :"))
display(Markdown(tutorial_response.final_output.tutorial))
```

```
draw_graph(outline_builder_agent)
```

```
outline_builder_agent = Agent(
    name= "Outline builder agent",
    instructions="""
        Given a particular programming topic, your job is to generate a tutorial by crafting an outline
        After making the outline, hand it to the tutorial generator agent"
    """,
    handoffs=[tutorial_generator_agent]
)
```

Outline :

1. Introduction

- Pour
- Typ

2. La boucle

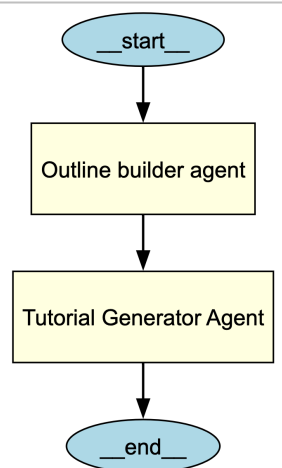
- Syn
- Exe
- Util

3. La boucle

- Syn
- Exe
- Cor

4. La boucle

- Syn
- Exemple simple
- Différences avec `while`



Triage Agent

```
from agents import Agent, handoff, RunContextWrapper, function_tool
from agents.extensions.visualization import draw_graph
```

```
@function_tool
def get_teacher_name(course : str):
    """
    Get The teacher name if course (Mathematics, History)
    """
    (parameter) course: str
    if(course=='Mathe
        return "Pr. H
    else:
        return "Pr. Najlaa"
```

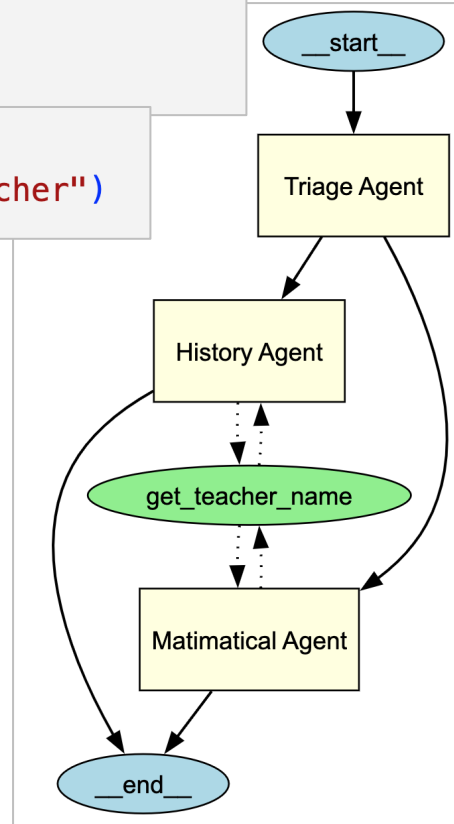
```
trriage_agent = Agent(
    name= "Triage Agent",
    instructions="""
        Answer the user question by delegating the task to the aproppriate agent,
        If neither agent is relevant, give a general response
    """,
    handoffs=[history_agent, math_agent]
)
```

```
trial_response = await Runner.run(trriage_agent,
    "How do i multiply X by y and give me the name of teacher")
```

```
history_agent = Agent(
    name= "History Agent",
    handoff_description= "Specialist Agent for historical questions",
    instructions="""
        You provide asistant with the historical queries
        Explain important events and context clearly
    """,
    tools= [get_teacher_name]
)
```

```
math_agent = Agent(
    name= "Matimatical Agent",
    handoff_description= "Specialist Agent for matimatical questions",
    instructions= """
        You provide asistant with the matimatical queries
        Explain your reasoning at each step and include examples
    """,
    tools= [get_teacher_name]
)
```

```
draw_graph(trriage_agent)
```



Triage Agent

```
from agents import Agent, handoff, RunContextWrapper, function_tool
from agents.extensions.visualization import draw_graph
```

```
@function_tool
def get_teacher_name(course : str):
    """
    Get The teacher name if course (Mathematics, History)
    """
    (parameter) course: str
    if(course=='Mathe
        return "Pr. H
    else:
        return "Pr. Najlaa"
```

```
triage_agent = Agent(
    name= "Triage Agent",
    instructions="""
        Answer the user question by delegating the task to the aproppriate agent,
        If neither agent is relevant, give a general response
    """,
    handoffs=[history_agent, math_agent]
)
```

```
trial_response = await Runner.run(triage_agent,
    "How do i multiply X by y and give me the name of teacher")
```

```
history_agent = Agent(
    name= "History Agent",
    handoff_description= "Specialist Agent for historical questions",
    instructions="""
        You provide asistant with the historical queries
        Explain important events and context clearlly
    """,
    tools= [get_teacher_name]
)
```

```
math_agent = Agent(
    name= "Matimatical Agent",
    handoff_description= "Specialist Agent for matimatical questions",
    instructions= """
        You provide asistant with the matimatical queries
        Explain your reasoning at each step and include examples
    """,
    tools= [get_teacher_name]
)
```

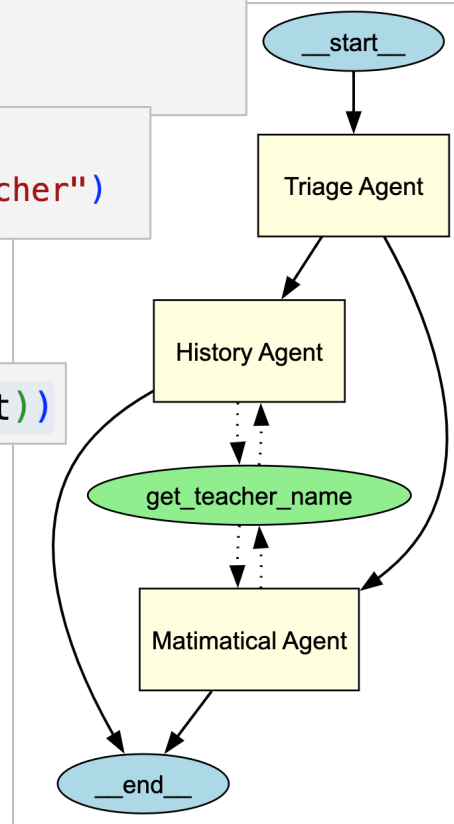
```
draw_graph(triage_agent)
```

```
display(Markdown(trial_response.final_output))
```

To multiply two numbers, (x) and (y), follow these steps:

- Understand the Multiplication:** Multiplication is a shortcut for repeated addition.
- Perform the Multiplication:**
 - If both (x) and (y) are positive integers, multiply them directly.
 - For example, if (x = 3) and (y = 4), then (3 \times 4 = 12).
- Use a Calculator for Complex Numbers:**
 - For large or decimal numbers, you might want to use a calculator.
- Special Properties:**
 - Zero Property:** Any number multiplied by 0 is 0.
 - E.g., (5 \times 0 = 0).
 - Identity Property:** Any number multiplied by 1 remains unchanged.
 - E.g., (7 \times 1 = 7).

The Mathematics teacher is Pr. Hassan.



Triage Agent

```
def on_history_handoff(ctx : RunContextWrapper[None]):
    print ("="*60)
    print ("History Agent is Invoqued")
    print ("="*60)

def on_math_handoff(ctx : RunContextWrapper[None]):
    print ("="*60)
    print ("Math Agent is Invoqued")
    print ("="*60)
```

```
trriage_agent = Agent(
    name= "Triage Agent",
    instructions="""
        Answer the user question by delegating the task to the apropriate agent,
        If neither agent is relevant, give a general response
    """,
    handoffs=[
        handoff(history_agent, on_handoff=on_history_handoff),
        handoff(math_agent, on_handoff=on_math_handoff)
    ]
)
```

```
trial_response = await Runner.run(trriage_agent,"HISTORY OF Rabat")
```

=====

History Agent is Invoqued

=====

```
trial_response = await Runner.run(trriage_agent,"C'est quoi l'ENSET")
display(Markdown(trial_response.final_output))
```

L'ENSET (École Normale Supérieure de l'Enseignement des disciplines techniques et professionnelles. Elle est chargée de former les futurs enseignants à transmettre des connaissances et les programmes peuvent inclure des études théoriques

```
trial_response = await Runner.run(trriage_agent,"How do i multiply X by y")
```

```
*****
Math Agent is Invoqued
*****
```

Agent Traces

<https://platform.openai.com/traces>

```
from agents import trace
```

```
with trace("Triage Work Flow"):
    trial_response = await Runner.run(triage_agent, "Hitoire de Rabat")
    trial_response.final_output
```

```
=====
History Agent is Invoqued
=====
```

< Traces / MCP Filesystem Example  trace_71e50a1f348640...			 Refresh	
Assistant		10.82 s		Properties
	List MCP Tools	3 ms		ID
	POST /v1/responses	3,422 ms		Workflow name
	write_file	2 ms		MCP Filesystem Example
	POST /v1/responses	1,361 ms		Metadata
	list_allowed_directories	2 ms		No metadata entries
	POST /v1/responses	2,944 ms		
	write_file	3 ms		
	POST /v1/responses	3,082 ms		

Agent Streaming

```
from openai.types.responses import ResponseTextDeltaEvent
joke_agent = Agent(
    name="My Agent",
    model='gpt-4o',
    instructions="You are a joke teller. you are giving a topic and you will tell 10 jokes about it"
)
result = Runner.run_streamed(joke_agent, "Food")
async for event in result.stream_events():
    if event.type=='raw_response_event' and isinstance(event.data, ResponseTextDeltaEvent):
        print(event.data.delta, end="", flush=True)
```

Sure thing! Here are 10 food jokes for you:

1. Why did the tomato turn red?
Because it saw the salad dressing!
2. What did the taco say to the burrito?
"I'm nacho friend anymore!"
3. Why don't eggs tell jokes?
They might crack up!
4. What do you call cheese that's not yours?
Nacho cheese!

Agent MCP : Model Context Protocol

```
## Create agent_mcp.py File
## Then Execute this application in console : > python
agent_mcp.py
import asyncio
import os
import shutil
from agents import Agent, Runner, gen_trace_id, trace
from agents.mcp import MCPServer, MCPServerStdio
from dotenv import load_dotenv

load_dotenv()

async def run(mcp_server: MCPServer):
    agent = Agent(
        name="Assistant",
        instructions="Use the tools to read the filesystem and
answer questions based on those files.",
        mcp_servers=[mcp_server],
    )

    # List the files it can read
    message = "Read the files and list them."
    print(f"Running: {message}")
    result = await Runner.run(starting_agent=agent, input=message)
    print(result.final_output)
```

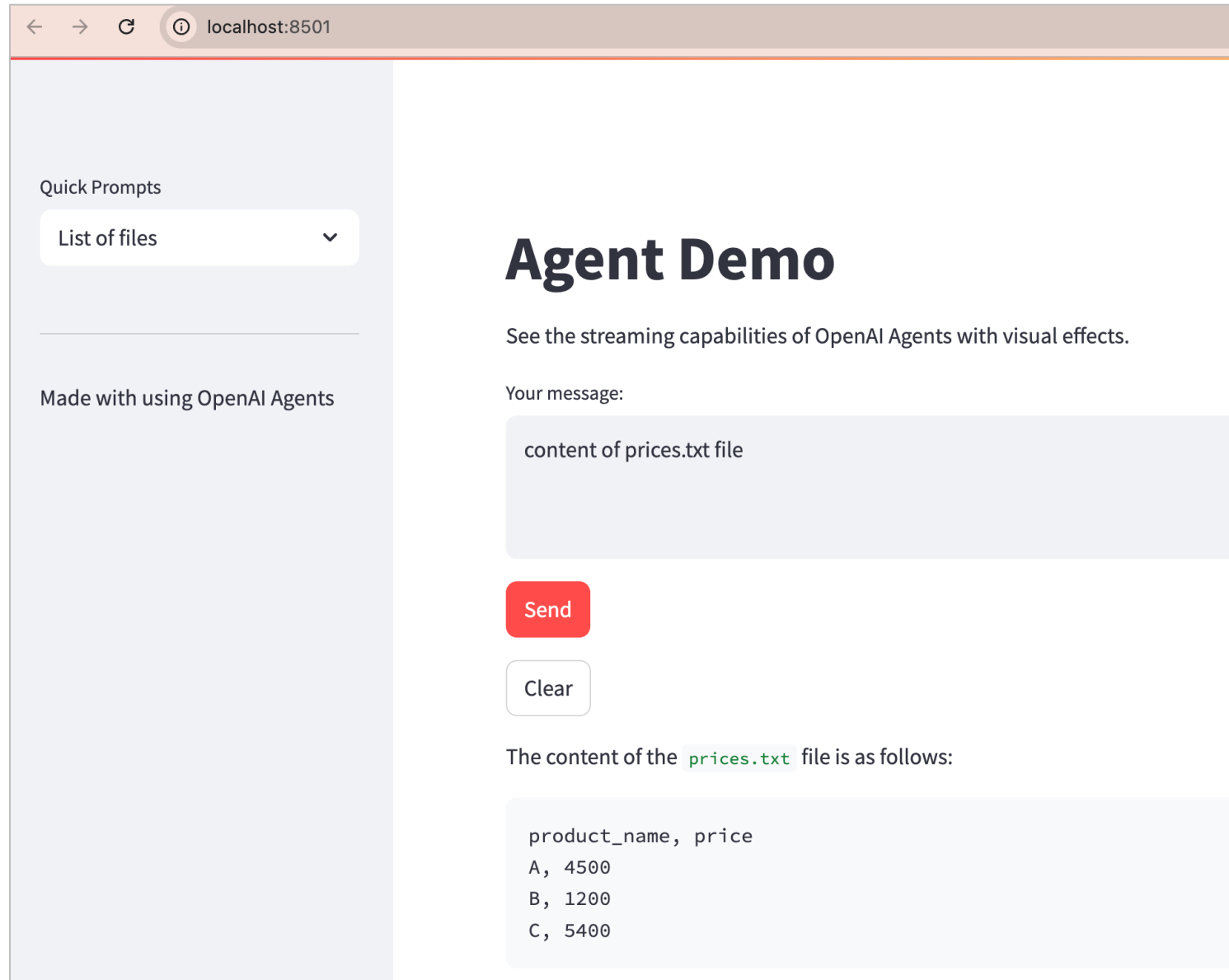
```
# Ask about books
message = "What is my #1 favorite book?"
print(f"\n\nRunning: {message}")
result = await Runner.run(starting_agent=agent, input=message)
print(result.final_output)

# Ask a question that reads then reasons.
message = "Look at my favorite songs. Suggest one new song that I might
like."
print(f"\n\nRunning: {message}")
result = await Runner.run(starting_agent=agent, input=message)
print(result.final_output)

async def main():
    current_dir = os.path.dirname(os.path.abspath(__file__))
    samples_dir = os.path.join(current_dir, "sample_files")
    os.makedirs(samples_dir, exist_ok=True)
    async with MCPServerStdio(
        name="Filesystem Server, via npx",
        params={
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-filesystem"],
        },
        samples_dir,
    ) as server:
        trace_id = gen_trace_id()
        with trace(workflow_name="MCP Filesystem Example", trace_id=trace_id):
            print(
                f"View trace:
https://platform.openai.com/traces/trace?trace_id={trace_id}\n"
            )
            await run(server)

if __name__ == "__main__":
    # Let's make sure the user has npx installed
    if not shutil.which("npx"):
        raise RuntimeError(
            "npx is not installed. Please install it with `npm install -g npx`."
        )
    asyncio.run(main())
```

Agent MCP : Model Context Protocol with Streamlit UI



Agent MCP : Model Context Protocol with Streamlit UI

```
import asyncio
import streamlit as st
from openai.types.responses import ResponseTextDeltaEvent
from agents.mcp import MCPServer, MCPServerStdio, MCPServerSse
import sys
import os
from dotenv import load_dotenv
load_dotenv()
api_key = os.environ.get("OPENAI_API_KEY")
if not api_key:
    raise ("API KEY not available")
# Add the parent directory to the path so we can import the agents
module
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from agents import Agent, Runner
st.set_page_config(
    page_title="Agent Demo",
    page_icon=".",
    layout="wide",
)
st.title("Agent Demo")
st.write("See the streaming capabilities of OpenAI Agents with visual effects.")
```

```
with st.sidebar:
    demo_options = [
        "List of files",
        "if not created, create new file data.txt",
        "content of data.txt file",
        "Look at my favorite songs. Suggest one new song that I might like.",
    ]
    demo_prompt = st.selectbox("Quick Prompts", demo_options)
    st.markdown("----")
    st.markdown("Made with using OpenAI Agents")
# User input area
user_input = st.text_area("Your message:", value=demo_prompt, height=100)
send_button = st.button("Send", type="primary")
# Response area with custom styling
response_container = st.container()
message_placeholder = st.empty()
```

Agent MCP : Model Context Protocol with Streamlit UI

```
import asyncio
import os
import shutil

from agents import Agent, Runner, gen_trace_id, trace
from agents.mcp import MCPServer, MCPServerStdio
from dotenv import load_dotenv

load_dotenv()

async def run(mcp_server: MCPServer):
    agent = Agent(
        name="Assistant",
        instructions="answer the user question using provided tools",
        mcp_servers=[mcp_server],
    )

    if send_button and user_input:
        with response_container:
            with st.spinner("Thinking..."):
                result = await Runner.run(starting_agent=agent,
input=user_input)

                print(result.final_output)
                message_placeholder.markdown(result.final_output)

            # Add clear button
            if st.button("Clear"):
                st.experimental_rerun()
```

```
# Instructions
if not send_button:
    with response_container:
        st.info("👉 Enter your message above and click 'Send' to see the streaming response.")
        st.markdown(
            """
            ### Tips:
            - Choose from the quick prompts or enter your own
            """
        )

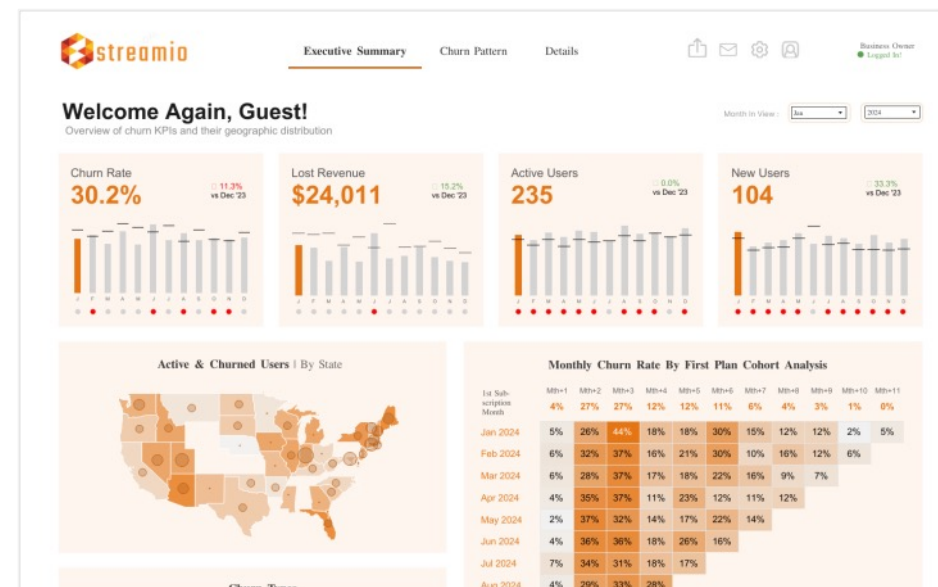
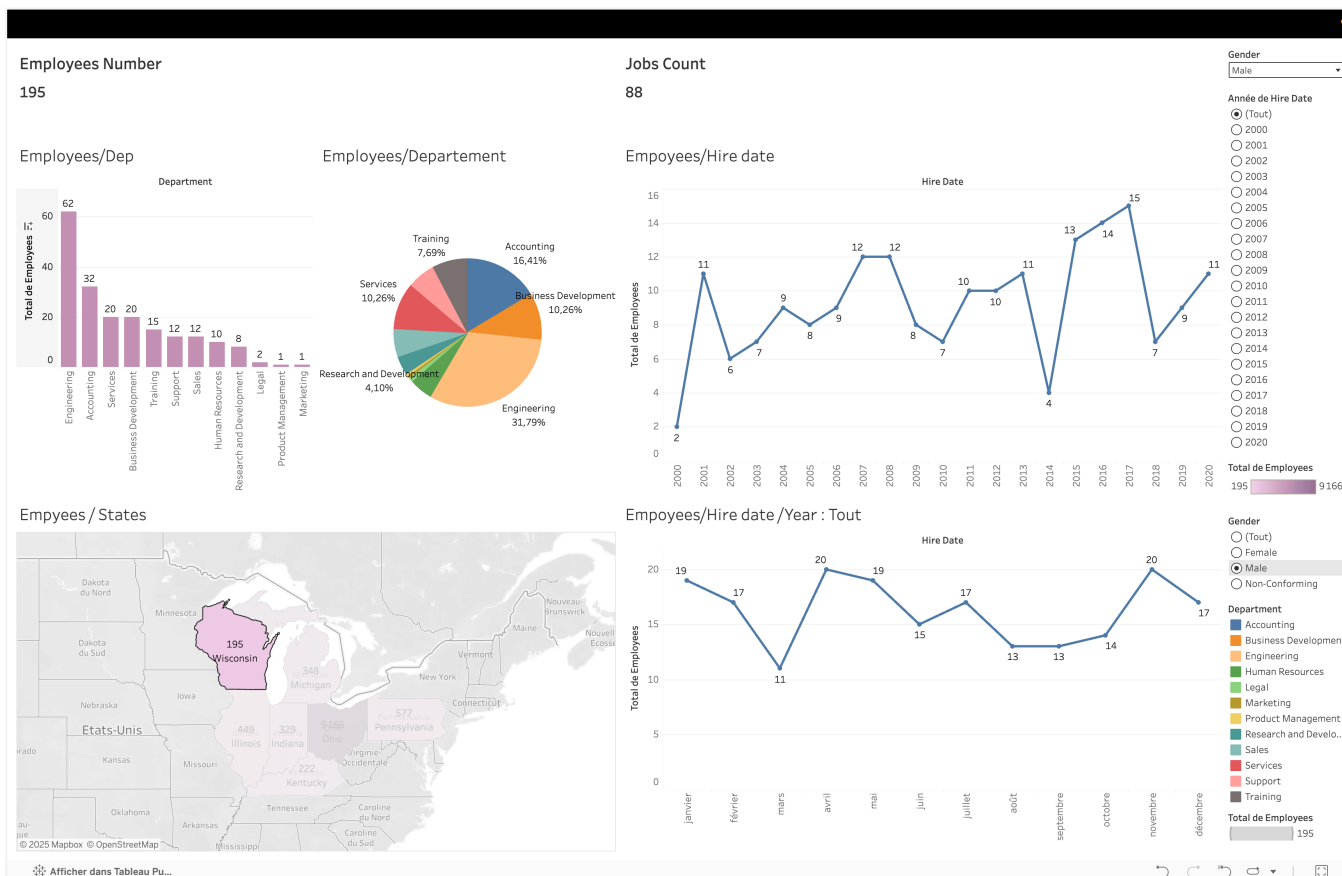
async def main():
    current_dir = os.path.dirname(os.path.abspath(__file__))
    samples_dir = os.path.join(current_dir, "sample_files")
    os.makedirs(samples_dir, exist_ok=True)
    async with MCPServerStdio(
        name="Filesystem Server, via npx",
        params={
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-filesystem", samples_dir],
        },
    ) as server:
        trace_id = gen_trace_id()
        with trace(workflow_name="MCP Filesystem Example", trace_id=trace_id):
            print(
                f"View trace: https://platform.openai.com/traces/trace?trace_id={trace_id}\n"
            )
            await run(server)

if __name__ == "__main__":
    # Let's make sure the user has npx installed
    if not shutil.which("npx"):
        raise RuntimeError(
            "npx is not installed. "
        )

    asyncio.run(main())
```

Data Analytics With Tableau Public

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité

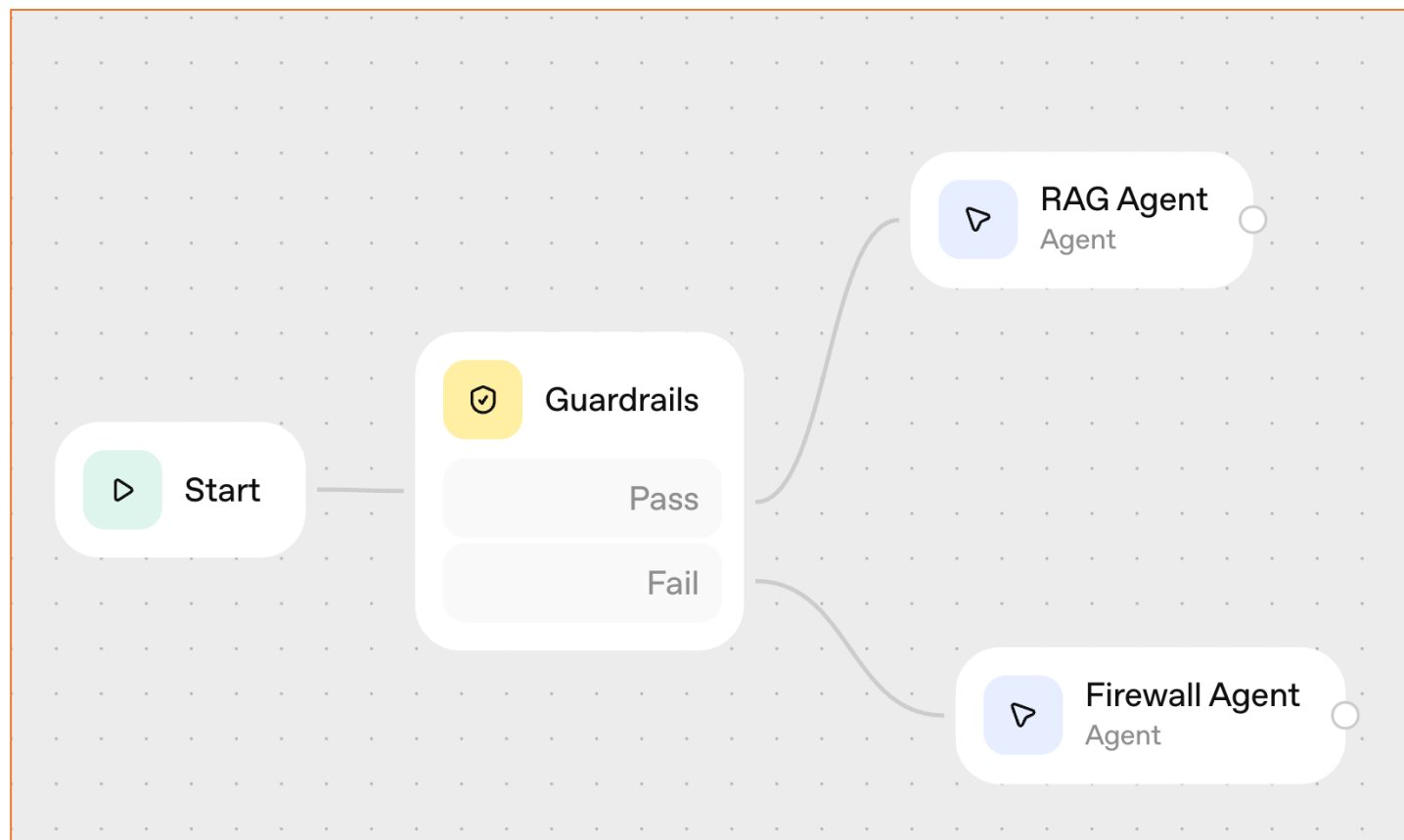


Agentic AI and Workflow Automation

Mohamed Youssfi, Enseignant Chercheur, ENSET Mohammedia, Directeur Laboratoire Informatique, Intelligence Artificielle et Cyber Sécurité



Open AI Agent Builder



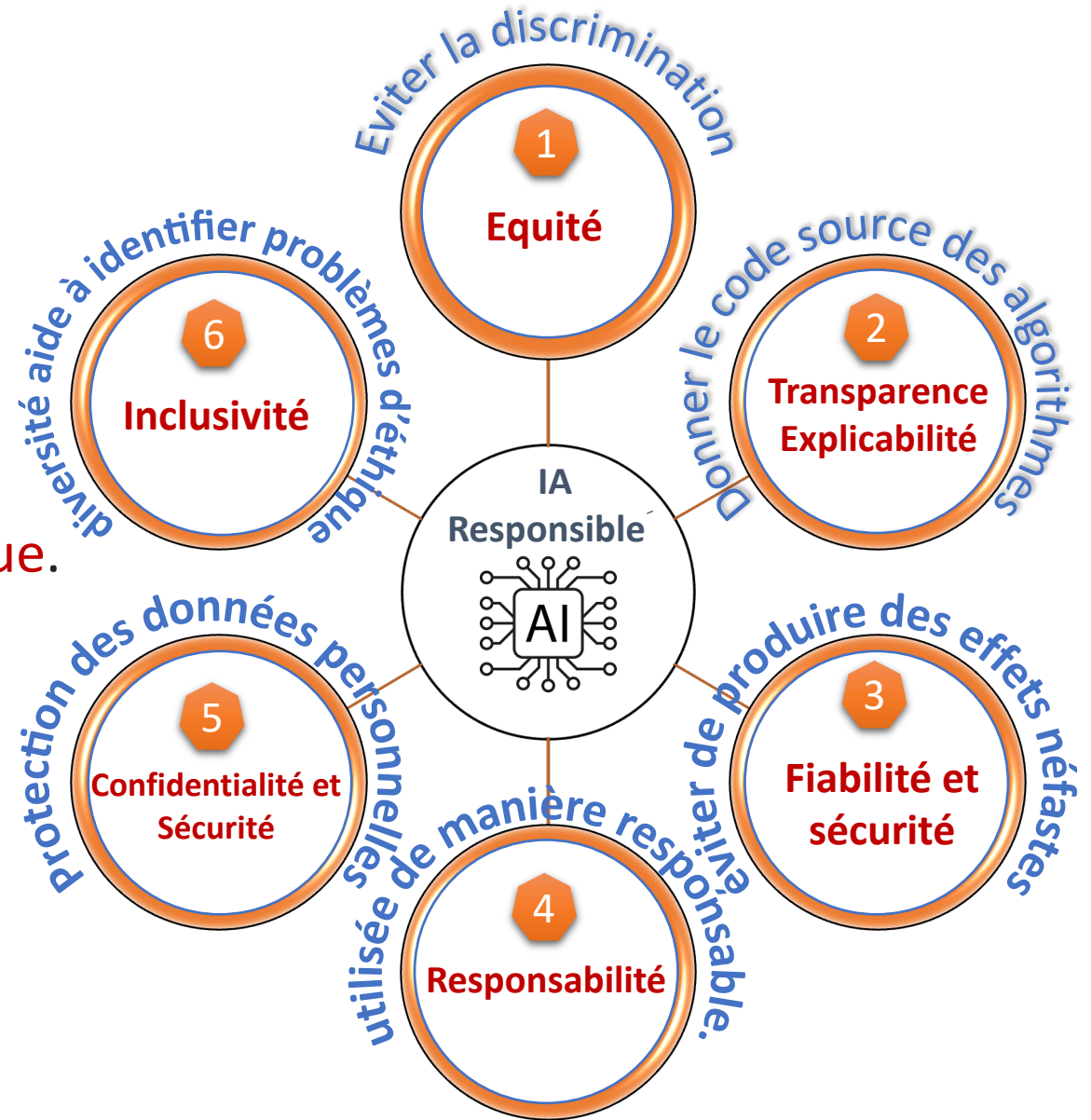
Facing a crazy world of agents

Principals of Responsible AI

L'IA

responsable est
une approche du
développement et du
déploiement de l'intelligence artificielle
sous un angle éthique et juridique.

L'objectif consiste à employer l'IA d'une
manière qui soit **sûre**,
fiable et **éthique**. Ainsi,
son utilisation gagnerait en
transparence et contribuerait à réduire les
problèmes tels que les biais (التحيزات).



Interact with LLMs : Open AI

<https://platform.openai.com/api-keys>

```
from langchain_openai import ChatOpenAI
```

```
OPEN_AI_KEY = "sk-proj-bkkDQ0UCtyf7Ctj2afRXT3BlbkFJtxzf8IbMmA6uqAd1qt9j"

gpt4_llm = ChatOpenAI(model="gpt-4o", temperature=0, api_key=OPEN_AI_KEY)

system_message = """
Classify the sentiment of the review presented in the input as 'positive' or 'negative'
The review will be delimited by triple backticks that is ``` in the input.
Answer only 'positive' or 'negative'
Do not explain your answer.
"""

user_message_template = "```{review}```"

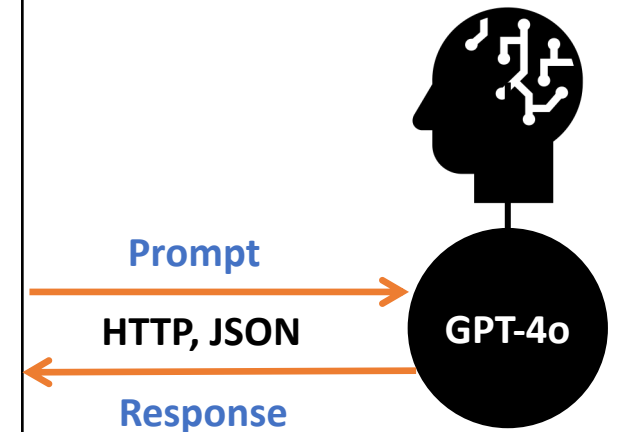
user_message = "I think that your services are very fine"

zero_shot_prompt = [
    {"role": "system", "content": system_message},
    {"role": "user", "content": user_message_template.format(review=user_message) },
]

response = gpt4_llm.invoke(zero_shot_prompt )

print(response.content)
```

Text Classification

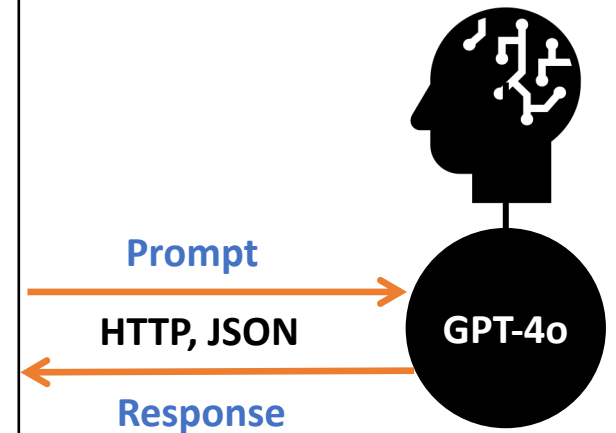


Interact with LLMs : Open AI- Direct Http Requests

```
import base64
import requests
# OpenAI API Key
api_key = "sk-proj-4f0aE2icZAzZ8qj1AvubT3BlbkFJqAsbp3Xx4qnvvIK5Jrj2"
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}
payload = {
    "model": "gpt-4o",
    "messages": [
        {"role": "system", "content": system_message},
        {"role": "user",
         "content": user_message_template.format(review=user_message)
        },
    ],
    "max_tokens": 300,
    "temperature": 0
}
url="https://api.openai.com/v1/chat/completions"
response = requests.post(url, headers=headers, json=payload)

print(response.json()[ 'choices' ][0][ 'message' ][ 'content' ])
```

Text Classification



Interact with LLMs : Groq Ollama Mistral

<https://console.groq.com/keys>

```
from langchain_groq import ChatGroq
```

```
GROQ_API_KEY = "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"

groq_llm = ChatGroq(model_name="llama3-70b-8192", temperature=0, api_key=GROQ_API_KEY)

system_message = """
    Classify the sentiment of the review presented in the input as 'positive' or 'negative'
    The review will be delimited by triple backticks that is ``` in the input.
    Answer only 'positive' or 'negative'
    Do not explain your answer.
    """

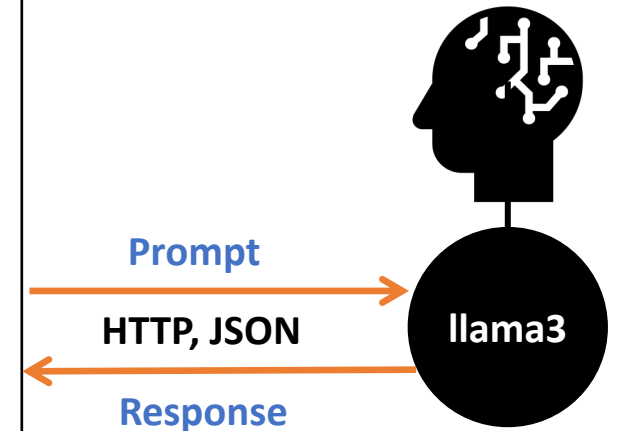
user_message_template = "```{review}```"

user_message = "I think that your services are very fine"

zero_shot_prompt = [
    {"role": "system", "content": system_message},
    {"role": "user", "content": user_message_template.format(review=user_message2)},
]

response = groq_llm.invoke(zero_shot_prompt )

print(response.content.replace("</s>", ""))
```



Interact with LLMs : Local Ollama LLM

```
from langchain_community.llms import Ollama
```

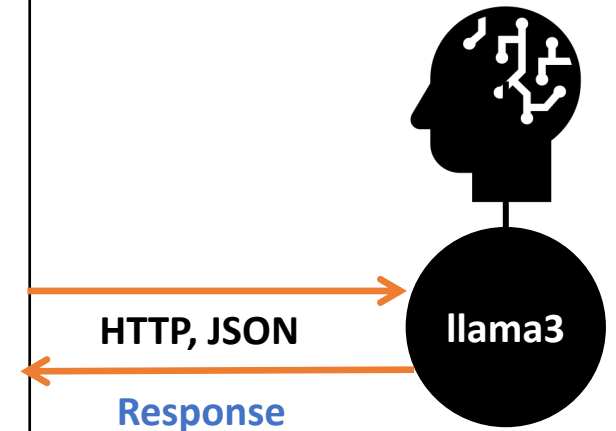
```
llama_llm = Ollama(model="llama3", temperature=0)

system_message = """
Classify the sentiment of the review presented in the input as 'positive' or 'negative'
The review will be delimited by triple backticks that is ``` in the input.
Answer only 'positive' or 'negative'
Do not explain your answer.
"""

user_message_template = "```{review}```"
user_message = "The look is bad"

few_shot_prompt = [
    {"role": "system", "content": system_message},
    {"role": "user", "content": user_message_template.format(review=user_message)},
]
response = llama_llm.invoke(few_shot_prompt )
print(response)
```

Llama3
in Local Machine



Interact with LLMs : Multi Model – Text + Image

```
import base64
import requests
api_key = "....."
# Function to encode the image
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode('utf-8')
image_path = "images/IMG_4479.jpg"
base64_image = encode_image(image_path)
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}
```

```
prompt_text = ""
```

Donne moi une description de cette image. Le résultat sera au format json avec les champs suivants :

- Description : Une description de l'image
- Nombre de personnes dans l'image
- Nombre de filles
- Nombre de garçon

```
""
```

```
payload = {
```

```
    "model": "gpt-4o",
```

```
    "messages": [
```

```
        {"role": "user", "content": [
```

```
            {"type": "text", "text": prompt_text}, {"type": "image_url", "image_url": {
                "url": f"data:image/jpeg;base64,{base64_image}"
            }}
        ]
    },
```

```
    "max_tokens": 300
```

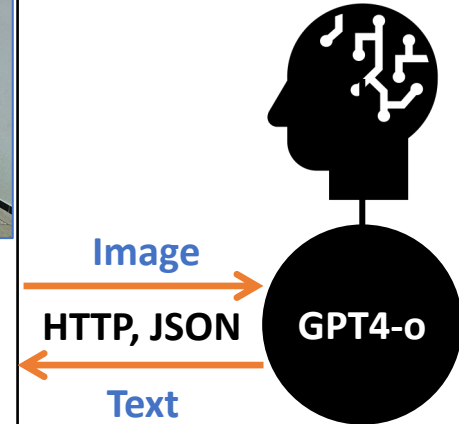
```
}
```

```
response = requests.post("https://api.openai.com/v1/chat/completions", headers=headers, json=payload)
```

```
print(response.json()['choices'][0]['message']['content'])
```



GPT-4o
Image Description



```
{
  "Description": "L'image montre une grande\nsalle de classe avec un total de 70\npersonnes. Elles sont assises à des bureaux\nen rangées, tournées vers l'objectif,\nsouriantes et semblant participer à une\nséance de cours. Les murs de la salle sont\nblancs et le plafond est équipé de\nluminaires modernes.",
  "Nombre de personnes dans l'image": 70,
  "Nombre de filles": 18,
  "Nombre de garçons": 52
}
```

Interact with LLMs : Image Generation – DALL-E

```
import json
import requests
import io
import base64
from PIL import Image
from IPython.display import display

url = "https://api.openai.com/v1/images/generations"

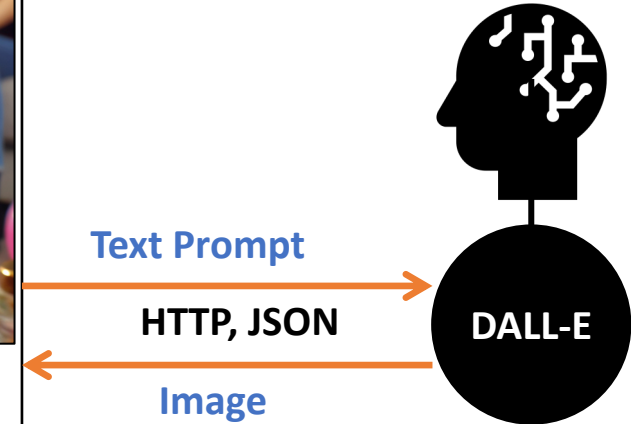
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}

payload = {
    "model": "dall-e-3",
    "prompt": "un chat avec un costume dans une fête avec un café dans sa main",
    "response_format": "b64_json",
    "n": 1,
    "size": "1024x1024"
}

response = requests.post(url=f'{url}', headers=headers, json=payload)
image_data = response.json()["data"][0]["b64_json"]
image = Image.open(io.BytesIO(base64.b64decode(image_data)))
image.save('output.png')
display(image)
```



GPT-4o
Image Description



Interact with LLMs : Image Generation – Stable Diffusion

```
import json
import requests
import io
import base64
from PIL import Image
from IPython.display import display

url = "http://127.0.0.1:7860"

payload = {
    "prompt": "Une peinture de PICASSO représentant un chat",
    "steps": 50,
    "width": 600,
    "height": 600
}

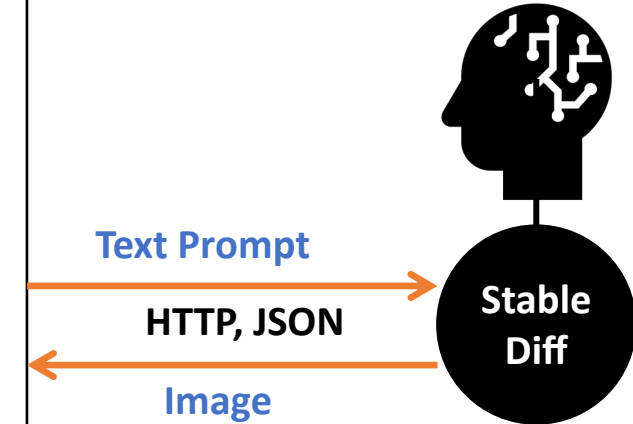
response = requests.post(url=f'{url}/sdapi/v1/txt2img', json=payload)

r = response.json()

image = Image.open(io.BytesIO(base64.b64decode(r['images'][0])))
image.save('output2.png')
display(image)
```

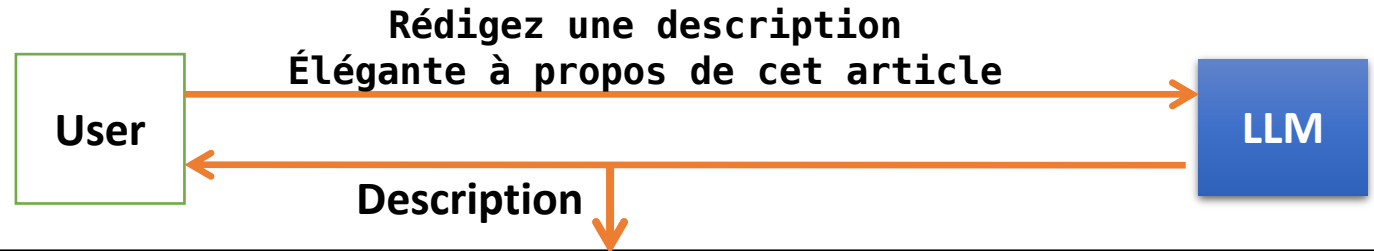
```
$ ./webui.sh --api
```

GPT-4o
Image Description



Instructional prompt (Prompt pédagogique)

- Les invites pédagogiques sont un partenaire précieux dans les tâches créatives où les idées initiales sont difficiles à trouver.
- Voici une invite qui accomplit cette tâche ; varier la température générera une grande variété de descriptions de produits facilitant le processus créatif.



```
instruction_prompt = ""
```

```
Vous êtes marketeur, Je voudrais mettre en vente ma voiture.  
Vous trouverez ci-dessous quelques informations sur cette voiture :
```

- **Marque : Toyota Corolla**
- **Modèle : 2017**
- **Couleur : Noir.**
- **Puissance fiscale : 8 cv**
- **Dernier prix : 120000**
- **Contact : 06442211 , infos@gmail.com**

```
Avec ces informations, rédigez une description élégante « À propos de  
cette voiture » qui sera utilisée sur un site de vente des voitures  
d'occasion.
```

```
Utilisez des puces pour délimiter les principales fonctionnalités  
mentionnées dans la description.
```

```
""
```

Reasoning Prompt with ReAct Framework

- Une meilleure méthode pour utiliser les capacités de raisonnement de GPT consiste à utiliser le Framework **ReAct (Reasoning and Action)**.
- Avec ce Framework, nous considérons GPT comme un agent intelligent et nous codifions explicitement les actions disponibles pour le modèle.
- Cela incite le modèle à peser sur les alternatives disponibles avant d'agir.

```
prompt = ""
```

```
Dans notre université, les enseignants ont constaté un réel problème chez les apprenants comme le manque de concentration, baisse du niveau des langues, des matières scientifiques et manque de discipline. L'équipe pédagogique a identifier quelques pistes pour résoudre ce problème comme :
```

- ```
1. La révisions des pratiques pédagogiques.
2. Instauration d'une pratique disciplinaire.
3. Révision des méthodes d'évaluation
4. Autres actions à identifier
```

```
Recommander un plan d'action et créer un plan étape par étape que l'université devra suivre dans cette situation.
```

```
Présentez votre réponse sous le format suivant :
```

```
{
 "question": <la question d'entrée à laquelle il faut répondre>,
 "reflection": <réfléchissez à vos options et au plan d'action pour chaque option>,
 "action": <l'action à entreprendre parmi les trois options présentées>,
 "raison": <le raisonnement derrière votre réponse>
 "étapes" : <plan étape par étape mettant en œuvre l'action>
}
```

# Induction Prompt

- Une approche pour générer des prompts pour un modèle de langage consiste à :
  - Présenter des paires d'entrées-sorties
  - et à inviter le modèle à générer un prompt qui obtient la sortie donnée à partir des entrées fournies.
- Cette technique peut servir de base d'idées d'incitation, contribuant ainsi à relancer le processus de génération de prompts.

Vous êtes un assistant qui aide l'équipe marketing d'une entreprise d'électronique à mieux comprendre les avis de ses clients. Les exemples d'entrées-sorties suivants ont été collectés à titre de suggestions pour que vous puissiez en tirer des leçons.

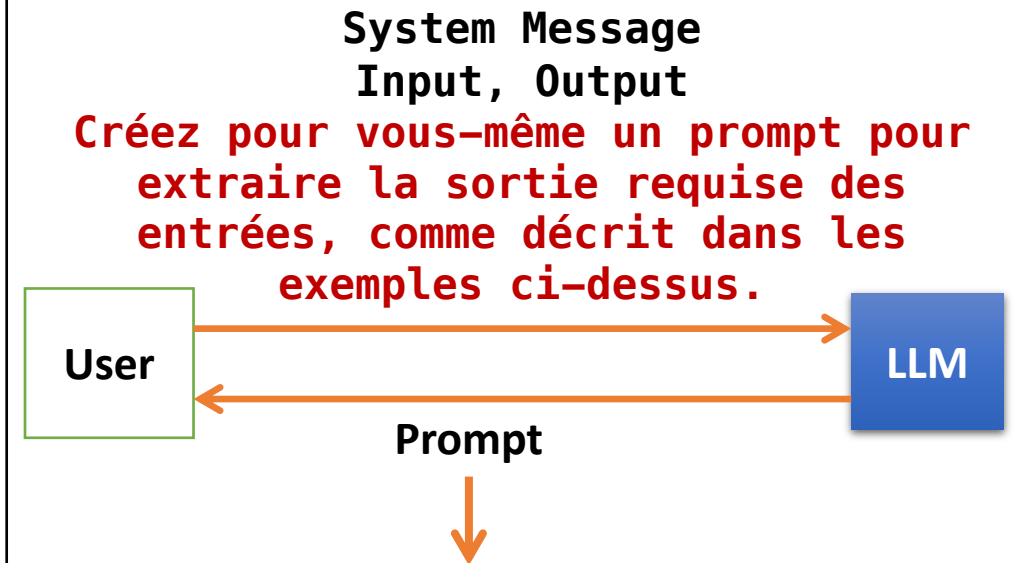
- **Entrée** : Gris commandé qui annonce un éclairage vert, quand on opte pour une esthétique cheap, c'est bouleversant. La -souris fonctionne bien.
- **Sortie** : **Souris**
- **Entrée** : j'en ai acheté un pour les jeux sur PC. J'ai adoré, puis j'en ai acheté une autre pour le travail. Cette souris n'est pas à la hauteur des souris haut de gamme comme la série Logitech MX Master, mais à 1/5-8ème du prix, je ne m'attendais pas à ce niveau de qualité. Il fonctionne bien, la molette de la souris semble lourde, les boutons latéraux sont bien placés avec différentes textures pour que vous puissiez les distinguer. La souris semble plutôt plastique et bon marché, mais pour le prix, c'est à peu près ce à quoi je m'attendais. J'aime une souris filaire pour éviter que le pointeur/le jeu ne saute à cause de la latence. Fil long également, ce qui minimise les problèmes d'accrochage. Très bon rapport qualité/prix dans l'ensemble.
- **Sortie** : **souris, Logitech MX Master, boutons DPI, molette de la souris, fil**

Créez pour vous-même un prompt pour extraire la sortie requise des entrées, comme décrit dans les exemples ci-dessus.

Créez l'invite pour qu'elle contienne des exemples généralisés à partir de ceux présentés ci-dessus.

N'oubliez pas que l'invite doit contenir des instructions que vous pouvez comprendre et générer le résultat attendu compte tenu de l'entrée.

""



## Prompt :

Lisez attentivement l'avis du client et identifiez le principal produit électronique en question. Notez également toutes les caractéristiques ou composants spécifiques du produit mentionnés, ainsi que tout autre produit électronique mentionné à des fins de comparaison. Le résultat doit être une liste de ces éléments.

# Paraphrasing Prompt

- Le Prompt de paraphrase est une technique qui consiste à générer un ensemble de prompts à l'aide d'un LLM à partir d'un Prompt de départ,
- Puis à évaluer les performances de chaque prompt sur un ensemble de test.
- Choisir le prompt le plus performant

```
paraphrase_prompt = """
```

```
Un prompt de départ vous sera présentée,
délimitée par trois backticks, c'est-à-dire
```
```

```
Ce prompt de départ sera présentée à un grand  
modèle de langage qui génère une sortie  
requis spécifique.
```

```
Veuillez générer 5 variantes du prompt de  
départ en gardant intacte l'intention de  
l'invite de départ.
```

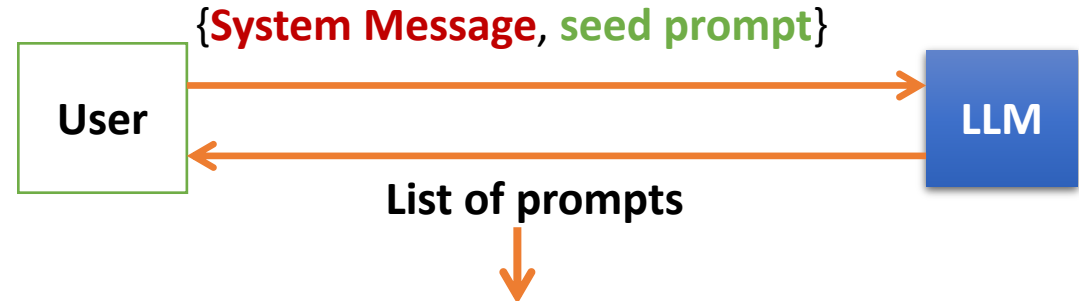
```
```
```

```
Veuillez analyser les avis clients suivants
et identifier les produits et fonctionnalités
qui sont mentionnés.
```

```
Fournissez une liste de produits et de
fonctionnalités mentionnés dans chaque avis.
```

```
```
```

```
"""
```



```
``` Analysez les commentaires des clients fournis ci-dessous et identifiez les produits et  
les fonctionnalités respectives qui sont discutés. Compilez une liste de ces produits et de
leurs fonctionnalités correspondantes mentionnées dans chaque avis. ```
```

```
``` Examinez les témoignages de clients fournis et déterminez les produits spécifiques ainsi  
que leurs caractéristiques auxquels il est fait référence. Créez une liste détaillant le  
produit et ses fonctionnalités associées mentionnées dans chaque avis. ```
```

```
``` Veuillez examiner les évaluations des clients ci-dessous et identifier les produits  
explicites et leurs fonctionnalités qui sont en discussion. Créez une liste des produits et
fonctionnalités mentionnés dans chaque avis individuel. ```
```

```
``` Pouvez-vous analyser les retours clients suivants et reconnaître les produits et les  
caractéristiques sur lesquels ils mettent l'accent ? Établissez une liste comprenant les  
produits et les fonctionnalités mentionnés dans chacun des avis. ```
```

```
``` Inspectez les évaluations des clients qui s'ensuivent et discerne les produits et leurs  
attributs qui sont mis en avant. Compilez une liste détaillant les produits et les
fonctionnalités respectives mentionnées dans chaque avis. ```
```

# CoT Prompt : Chain-of-Thought Prompting :

## Chain-of-Thought Prompting : CoT Prompt

- Pour l'invite CoT, nous ajoutons des instructions détaillées étape par étape au message système demandant au modèle de réfléchir attentivement avant de décider du résultat à générer
- En dehors de cet ajout, il n'y a aucun autre changement par rapport à prompt standard

```
cot_system_message = """"
Classify the sentiment of movie reviews presented
in the input as 'positive' or 'negative'.
Movie reviews will be delimited by triple backticks
in the input.
Answer only 'positive' or 'negative'. Do not
explain your answer.
```

### Instructions:

1. Carefully read the text of the review and think through the options for sentiment provided
2. Consider the overall sentiment of the review and estimate the probability of the review being positive

To reiterate, your answer should strictly only contain the label: positive or negative.

```
""""
```

## Standard Prompt

- Q : Ahmed has 5 tennis balls, he buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
- A : **The answer is 11**
- Q: **The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more. How many apples do they have?**

The cafeteria now has 9 apples.

## CoT Prompt

- Q : Ahmed has 5 tennis balls, he buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
- A : **Ahmed started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls.  $5+6=11$ . The answer is 11**
- Q: **The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more. How many apples do they have?**

The cafeteria started with 23 apples. They used 20, so they had  $23-20=3$  apples left. Then they bought 6 more, so they now have  $3+6=9$  apples.

# The building blocks of RAG

```
Importing the streamlit library to create the web app interface
import streamlit as st

Importing the PyPDF2 library to read the PDF files and extract the text from the PDF files
from PyPDF2 import PdfReader

Importing the CharacterTextSplitter class from the langchain library to split the text into chunks
from langchain.text_splitter import CharacterTextSplitter

Importing the OpenAIEmbeddings class from the langchain library to create the vector store
from langchain.embeddings import OpenAIEmbeddings

Importing the FAISS class from the langchain library to create the vector store
from langchain.vectorstores import FAISS

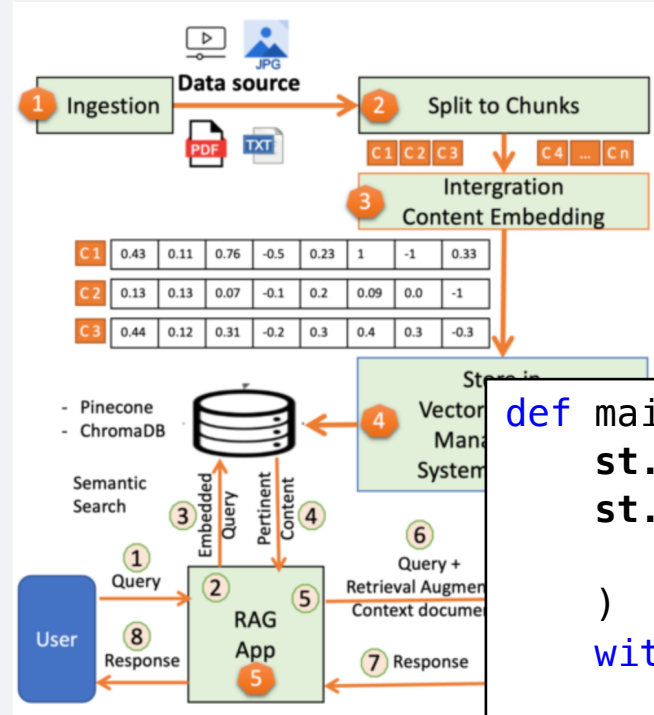
Importing the ChatOpenAI class from the langchain library to create the language model
from langchain.chat_models import ChatOpenAI

Importing the ChatPromptTemplate class from the langchain library to create the prompt
from langchain_core.prompts import ChatPromptTemplate

Importing the create_stuff_documents_chain and create_retrieval_chain functions from the langchain library
from langchain.chains.combine_documents import create_stuff_documents_chain

Importing the create_retrieval_chain function
from langchain.chains import create_retrieval_chain
from langchain_community.llms import Ollama
from langchain_community.embeddings.fastembed import FastEmbedEmbeddings
```

# The building blocks of RAG



Upload Your PDFs

Drag and drop files here

Limit 200MB per file

Browse files

 [tsla-20221231-gen.pdf](#)  
1.7MB

Submit

## Retrieval Augmented Generation (RAG) Pedagogical Chatbot

## Chatbot zone

Ask your question :

What is the company's business model?

The company's business model involves designing, developing, manufacturing, selling, and leasing high-performance fully electric vehicles and energy

```
def main():
 st.set_page_config(layout="wide")
 st.subheader(
 "Retrieval Augmented Generation"
)
 with st.sidebar:
 st.sidebar.title("Data Loader")
 st.image("rag.png", width=500)
 pdf_docs = st.file_uploader(
 label="Upload Your PDFs",
 accept_multiple_files=True,
)
 if st.button("Submit"):
 with st.spinner("Loading..."):
 # #####
```

```
st.subheader("Chatbot zone")
Sidebar of the web app
user_question = st.text_input("question :")
if user_question:
 response = "???????????"
 st.write(response)
if __name__ == "__main__":
 main()
```

# RAG : Extracting Text from PDFs

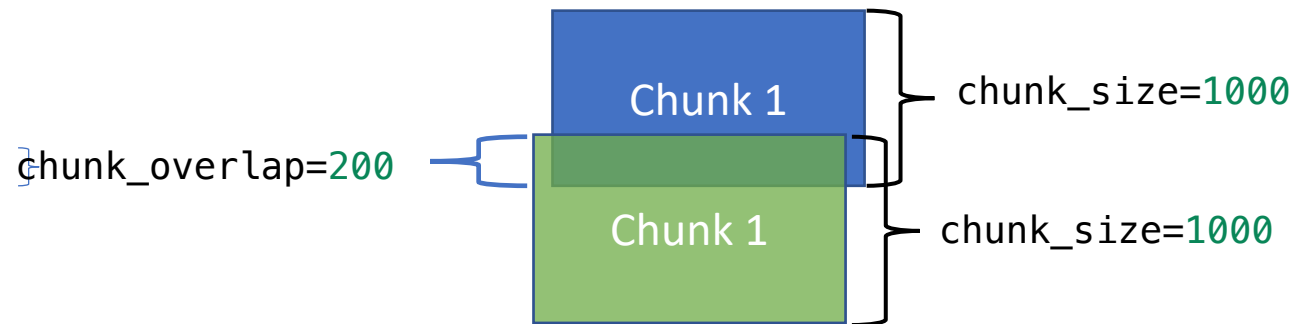
```
with st.sidebar:
 st.sidebar.title("Sidebar")
 pdf_docs = st.file_uploader("Upload PDF", accept_multiple_files=True)
 if st.button("Submit"):
 # Loading spinner to show the process is running
 with st.spinner("Loading..."):
 # Extract the content of the PDF
 pdf_content = ""
 # Loop through the PDF files
 for pdf in pdf_docs:
 # Read the PDF file
 pdf_reader = PdfReader(pdf)
 # Loop through the pages of the PDF file
 for page in pdf_reader.pages:
 # Extract the text from the PDF page and add it to the pdf_content variable
 pdf_content += page.extract_text()
 # st.write(pdf_content)
```

# RAG / Split Text into Chunks

```
Get chunks of the content
Split the text into chunks of 1000 characters with an overlap of 200 characters
text_splitter = CharacterTextSplitter(
 separator="\n",
 chunk_size=1000,
 chunk_overlap=200,
 length_function=len,
)

Split the text into chunks of 1000 characters with an overlap of 200 characters
chunks = text_splitter.split_text(pdf_content)

Display the chunks of the text
st.write(chunks)
```

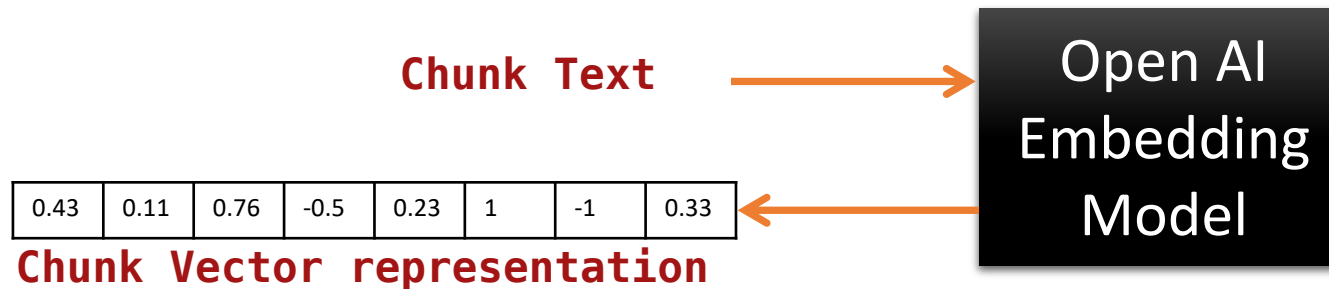


# RAG : Create Vector Store using Open AI Embedding

```
OpenAI API key
OPEN_API_KEY = "sk-....."

Create the OpenAIEmbeddings object
embedding_model = OpenAIEmbeddings(api_key = OPEN_API_KEY)
embedding_model = FastEmbedEmbeddings()

Create the FAISS vector store from the text chunks and the OpenAIEmbeddings object
openai_vector_store = FAISS.from_texts(
 texts=chunks, embedding = embedding_model
)
```

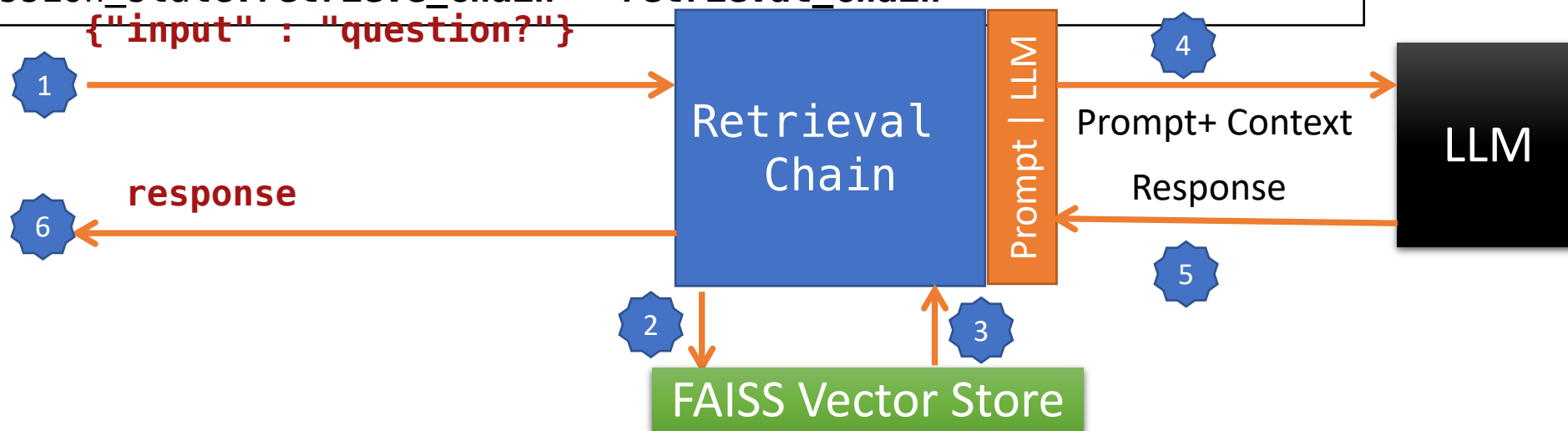


# RAG

```
llm = ChatOpenAI(api_key=OPEN_API_KEY)
llm = Ollama(model="llama3", temperature=0)
prompt = ChatPromptTemplate.from_template(
 """
 Answer the following question based only on the provided context:
 <context>
 {context}
 </context>
 Question: {input}
 """
)
document_chain = create_stuff_documents_chain(llm, prompt)
retriever = openai_vector_store.as_retriever()
retrieval_chain = create_retrieval_chain(retriever, document_chain)
st.session_state.retrieve_chain = retrieval_chain
```

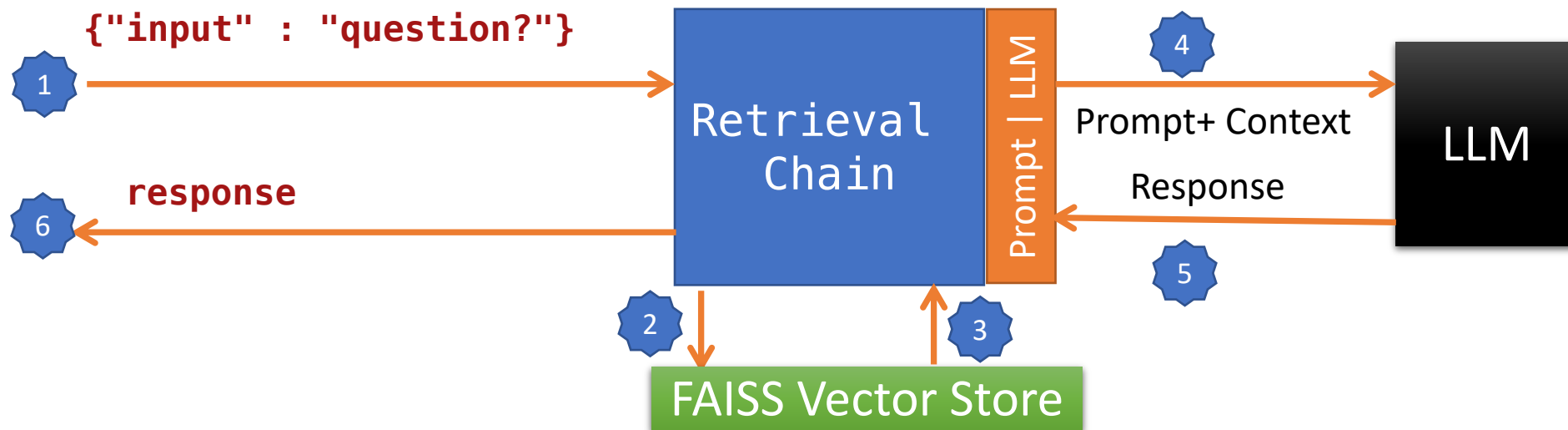
**1** {"input": "question?"}

- 1** User Query
- 2** Similarity Vector Search
- 3** Relevant Documents
- 4** Prompt LLM
- 5** LLM Response
- 6** RAG Response



# RAG

```
st.subheader("Chatbot zone")
user_question = st.text_input("Ask your question :")
if user_question:
 response = st.session_state.retrieve_chain.invoke({"input": user_question})
 st.write(response["answer"])
```



# Max Tokens

Max Tokens Limit : 4096



Prompt Max Tokens : 800

Model	Input	Output	Max tokens	Training data
gpt-3.5-turbo	\$0.0015 / 1K tokens	\$0.002 / 1K tokens	4,096 tokens	Up to Sep 2021
gpt-3.5-turbo-16k	\$0.003 / 1K tokens	\$0.004 / 1K tokens	16,384 tokens	Up to Sep 2021

Model	Input	Output	Max tokens	Training data
gpt-4	\$0.03 / 1K tokens	\$0.06 / 1K tokens	8,192 tokens	Up to Sep 2021
gpt-4-32k	\$0.06 / 1K tokens	\$0.12 / 1K tokens	32,768 tokens	Up to Sep 2021

<https://platform.openai.com/tokenizer>

GPT-3.5 & GPT-4   GPT-3 (Legacy)

You are an objective assistant, part of multinational computer, phone, laptop, and hardware manufacturer, to summarize unstructured customer reviews in simple words.

Tokens      Characters  
29            165

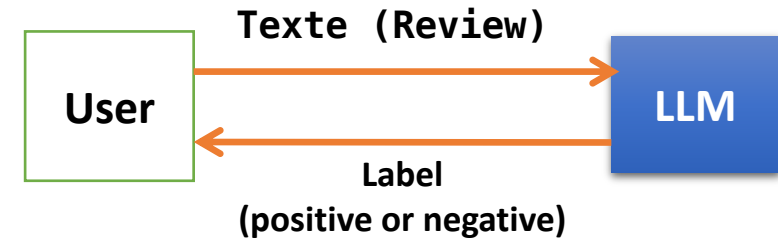
You are an objective assistant, part of multinational computer, phone, laptop, and hardware manufacturer, to summarize unstructured customer reviews in simple words.

[2675, 527, 459, 16945, 18328, 11, 961, 315, 69026, 6500, 11, 4641, 11, 21288, 11, 323, 12035, 14290, 11, 311, 63179, 653, 52243, 6130, 8544, 304, 4382, 4339, 13]

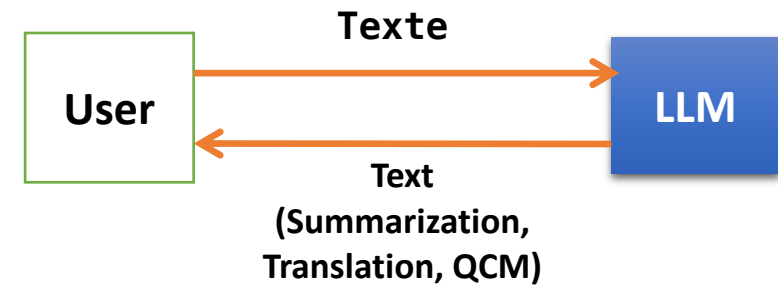
# Retrieval-Augmented Generation (RAG)

- Considérons que nous souhaitons interroger le LLM pour comprendre le contenu d'un rapport annuel.
- Bien que le modèle soit capable de comprendre les instructions contenues dans le prompt, il ignore parfaitement ce qui se passe après la date de son entraînement.
- Concrètement, il est fort probable que le rapport annuel n'ait pas été ingéré lors de l'entraînement.
- **Dans ce cas, le résultat de LLM serait, au mieux, inutile et, au pire, incorrect (appelé « Hallucination »).**
- Pour corriger cette sérieuse limitation, nous devons assembler et injecter un contexte généré dynamiquement en fonction des entrées de l'utilisateur.
- Un exemple typique est celui où les données stockées par les organisations sont utilisées comme source de contexte pour répondre aux questions posées par les utilisateurs.
- **Cette idée de récupération et d'injection d'un contexte pertinent pour la requête dans un prompt pour augmenter la génération de réponse est appelée Retrieval Augmented Generation (RAG)**

## Text to Label



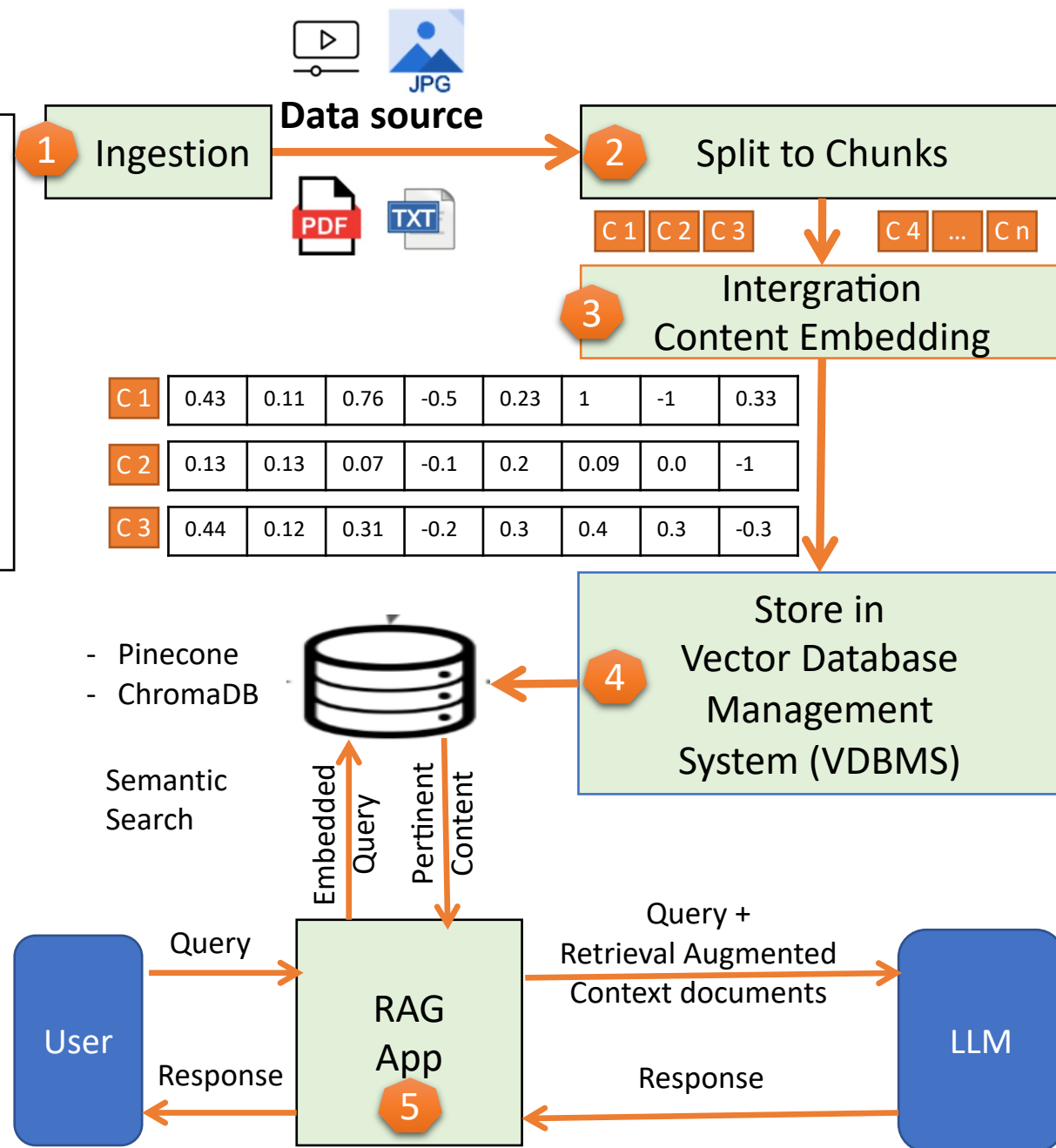
## Text to Text



# The building blocks of RAG

## Step 1 : Ingestion des sources de données

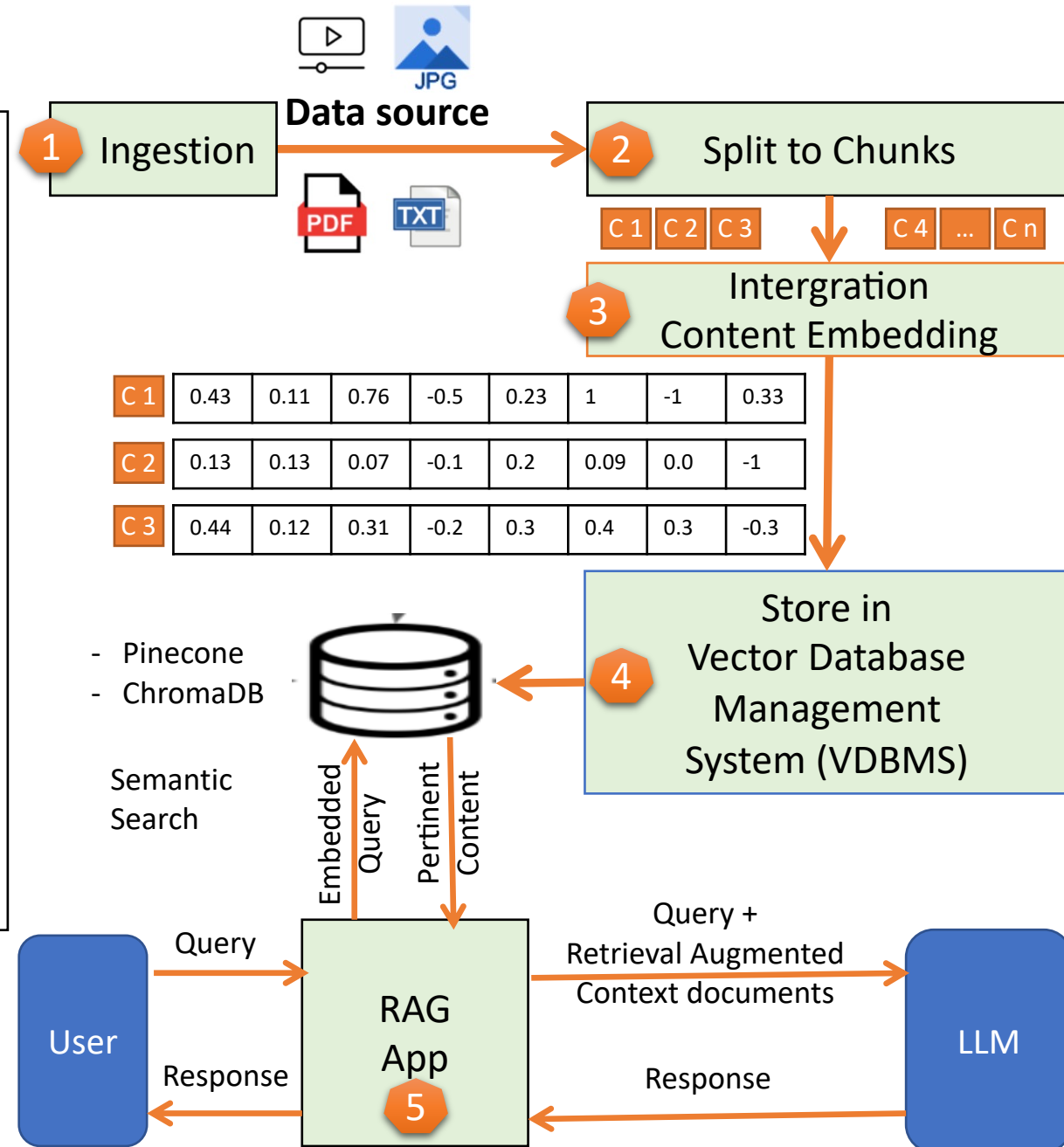
- Ingestion des différentes sources de données dans une base de données pour former une couche de contexte (PDF, Audio, Vidéos, Excel, Word, etc.).
- Nous devons sélectionner soigneusement les documents liés à la saisie de l'utilisateur afin que le LLM puisse être chargé de répondre uniquement en fonction du contexte.



# The building blocks of RAG

## Step 2 : Split to Chunks

- Transformer les données brutes en morceaux plus petits avant de les transmettre à un modèle d'intégration
- Deux raisons importantes pour lesquelles cela est nécessaire :
  - Les modèles d'embedding sont des modèles de transformateurs et ont donc des contraintes de longueur d'entrée.
  - Le découpage des données en morceaux facilite un contrôle précis des informations spécifiques qui peuvent être injectées comme contexte. Cela aide le modèle à se concentrer uniquement sur les informations les plus pertinentes pour la requête.

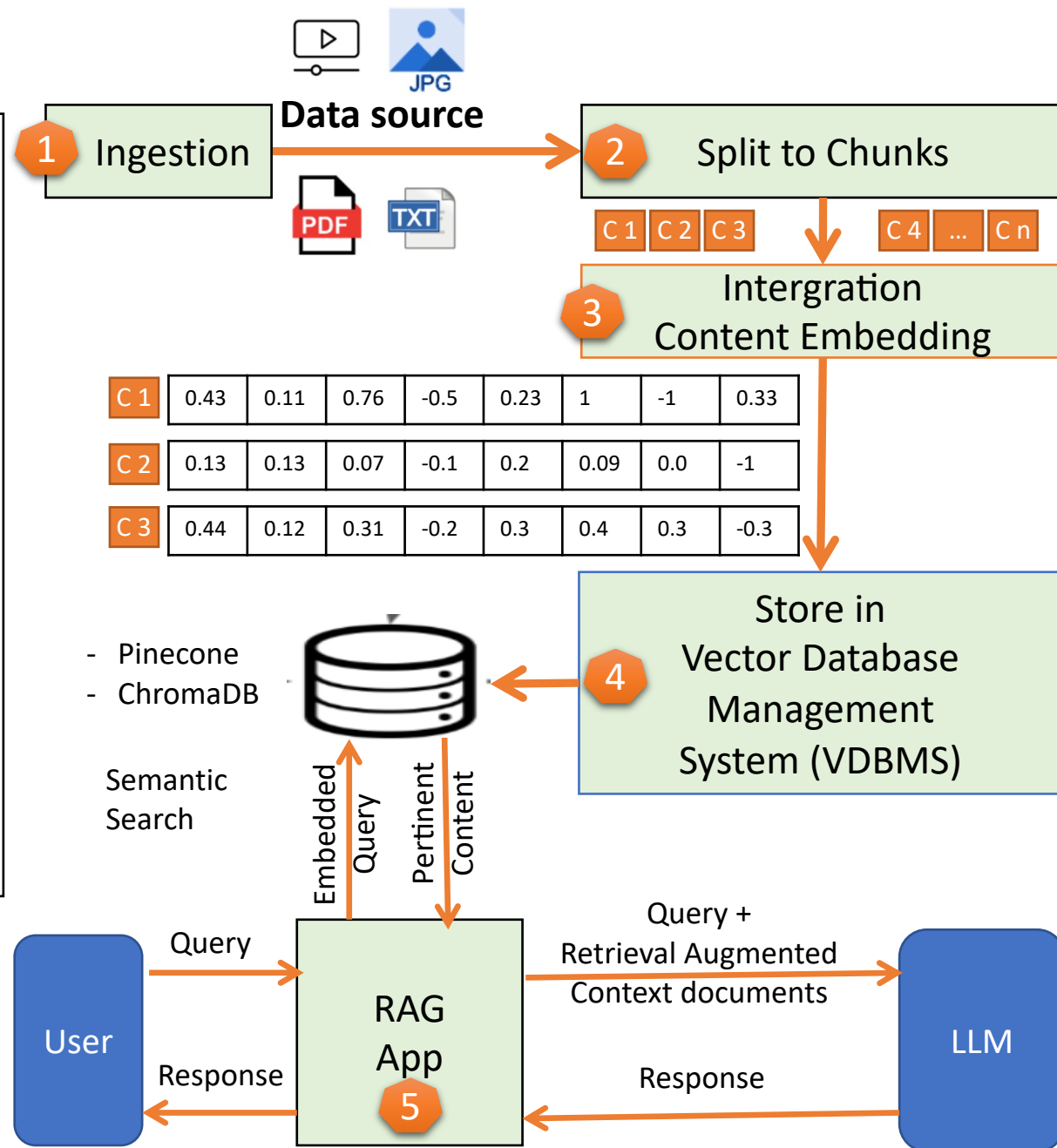


# The building blocks of RAG

## Step 3 : Content Embedding

- Transformer les données brutes des chunks en représentations vectorielles.
- Ceci permet de construire un moteur de recherche sémantique avec une base de donnée locale
- De cette façon, nous pouvons comparer la requête (convertie en vecteur) avec les vecteurs de documents et choisir uniquement les documents similaires à la requête.
- Cette recherche est basée sur une mesure de similarité comme **Cosine Similarity**
- Ce processus de création d'une représentation vectorielle à partir du texte d'entrée est appelé **intégration** et est réalisé à l'aide de **modèles d'intégration** (par exemple, Open AI `ada`).

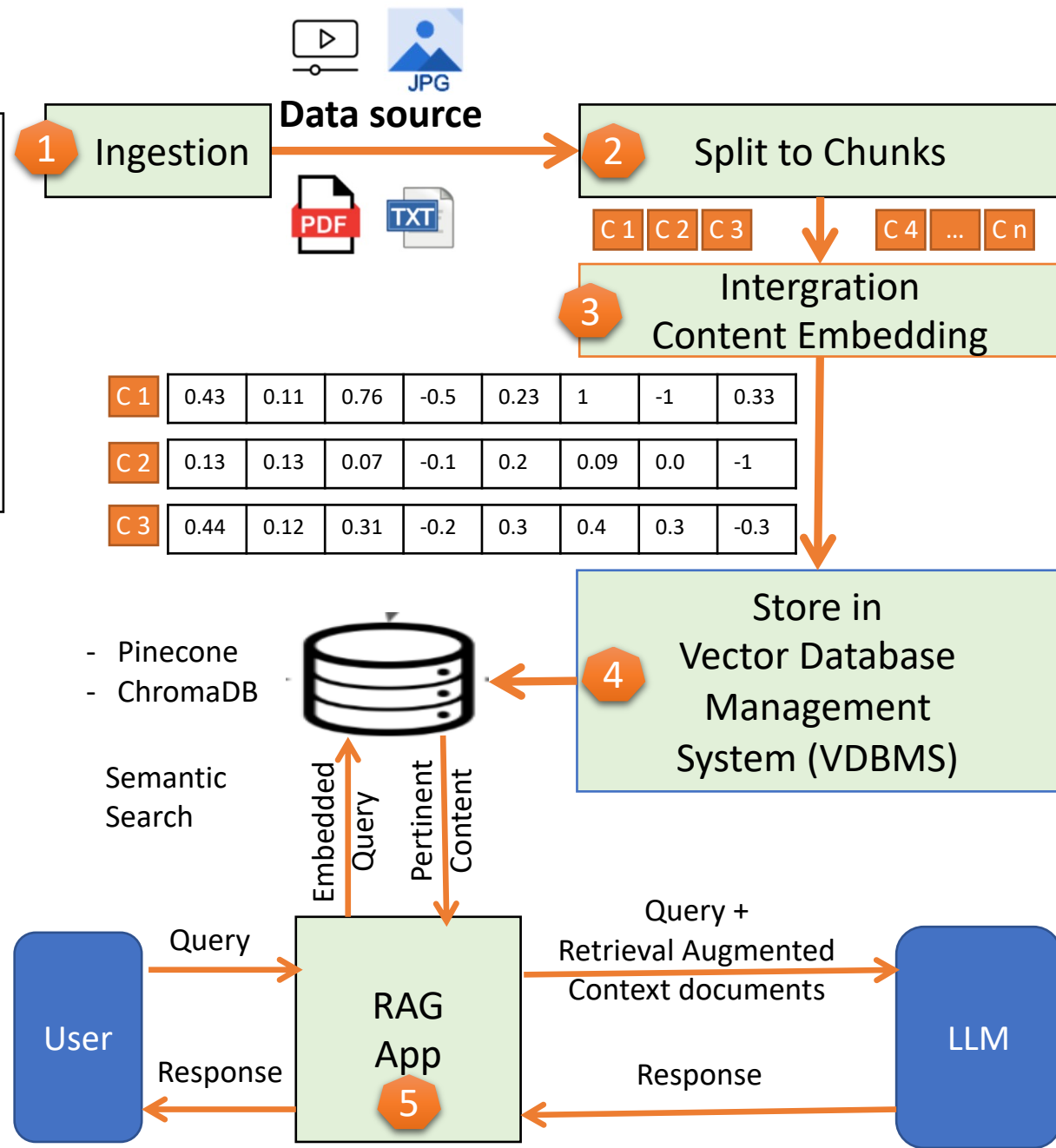
<https://huggingface.co/spaces/mteb/leaderboard>



# The building blocks of RAG

## Step 4 : Store in Vector Database (VDBMS)

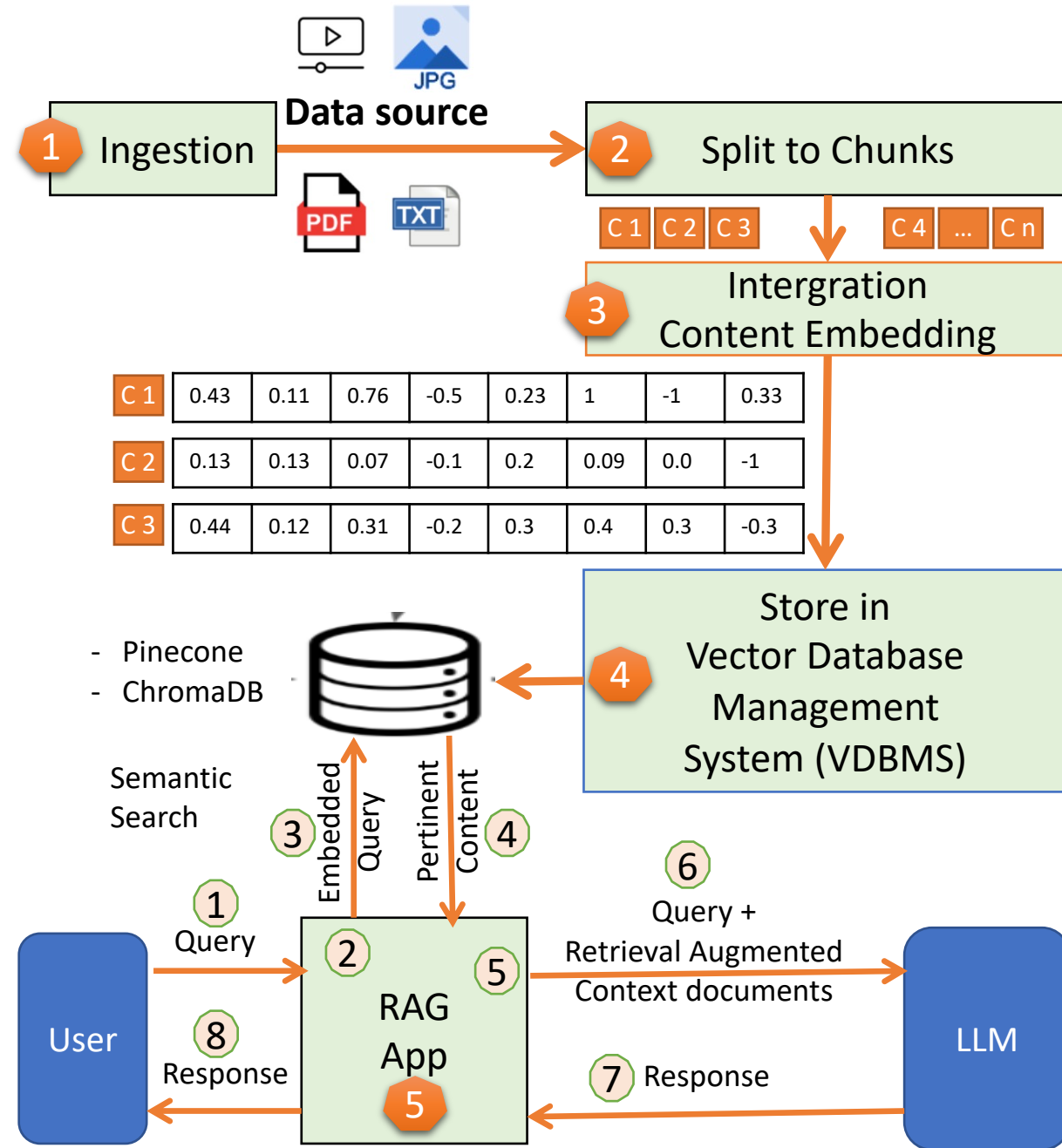
- Une fois les intégrations créées, elles peuvent être stockées pour une récupération efficace dans des magasins de données spécialisés tels que :
  - [Pinecone](<https://www.pinecone.io/>)
  - [ChromaDB](<https://www.trychroma.com/>).
  - PostgreSQL



# The building blocks of RAG

## Step 5 : RAG App

- Développer une application qui permet de :
  1. Saisir la requête utilisateur
  2. Transformer la requête en représentation vectorielle
  3. Interroger le VDBMS (Pinecone, ChromaDB, PostgreSQL) pour faire une recherche sémantique.
  4. Retrouver les documents pertinents lié au contexte de la requête
  5. Injecter les contenus contextuels retrouvés dans le prompt
  6. Présenter ce prompt au LLM pour l'inciter à réponse à la requête utilisateur en utilisant les données contextuelles les plus pertinentes.
  7. Récupérer la réponse du LLM
  8. Envoyer à l'utilisateur la réponse



# Demos : Open AI

<https://platform.openai.com/assistants>

The screenshot displays the OpenAI Playground web interface. The browser's address bar shows the URL `platform.openai.com/playground?mode=assistant&assistant=asst_0P3LErVtmq3PEY...`. The page title is "Playground - OpenAI API".

The interface is divided into two main sections. The left sidebar contains a vertical menu with icons for various features. The main content area is split into two panels:

- Configuration Panel (Left):**
  - Y11**: The name of the assistant, with a dropdown arrow.
  - Name**: A text input field containing "Y11". Below it, the assistant ID `asst_0P3LErVtmq3PEYmCrifCuPuw3` is displayed.
  - Instructions**: A text area containing the instruction: "Perform aspect based sentiment analysis on laptop reviews presented in the input delimited by triple backticks, that is, ```". Below this, it says "In each review there might be one or more of the".
  - Model**: A dropdown menu showing "gpt-4".
  - Actions**: At the bottom left, there are buttons for "Delete" (trash icon) and "Clone".
  - Updated**: At the bottom right, it says "Updated 16/02 11:25".
- Thread Panel (Right):**
  - THREAD**: The title of the current thread.
  - Clear** and **Logs**: Buttons at the top right of the thread panel.
  - Message Input**: A large text area with the placeholder "Enter your message...". Below it are buttons for "Attach" (paperclip icon), "Add" (+), and "Run" (play button icon).
  - Disclaimer**: At the bottom, it states "Playground messages can be viewed by anyone at your organization using the API."

# Demos : Huggingface

<https://huggingface.co/chat>

The screenshot displays the HuggingChat web interface. The browser's address bar shows the URL `huggingface.co/chat`. The interface is divided into a left sidebar and a main chat area.

**Left Sidebar:**

- HuggingChat** (with a smiley face icon) and a **New Chat** button.
- Today:** A chat titled "Student management app" with a laptop icon.
- This week:** Two chats: "Good screen, bad keyboard & m" (with a smiley face icon) and "Screen quality, keyboard & mou:" (with a smiley face icon).
- A chat titled "Sum function" with a globe icon.
- A section for **mohamedyousfi** containing:
  - Theme**
  - Assistants** (with a **New** button)
  - Settings**
  - About & Privacy**

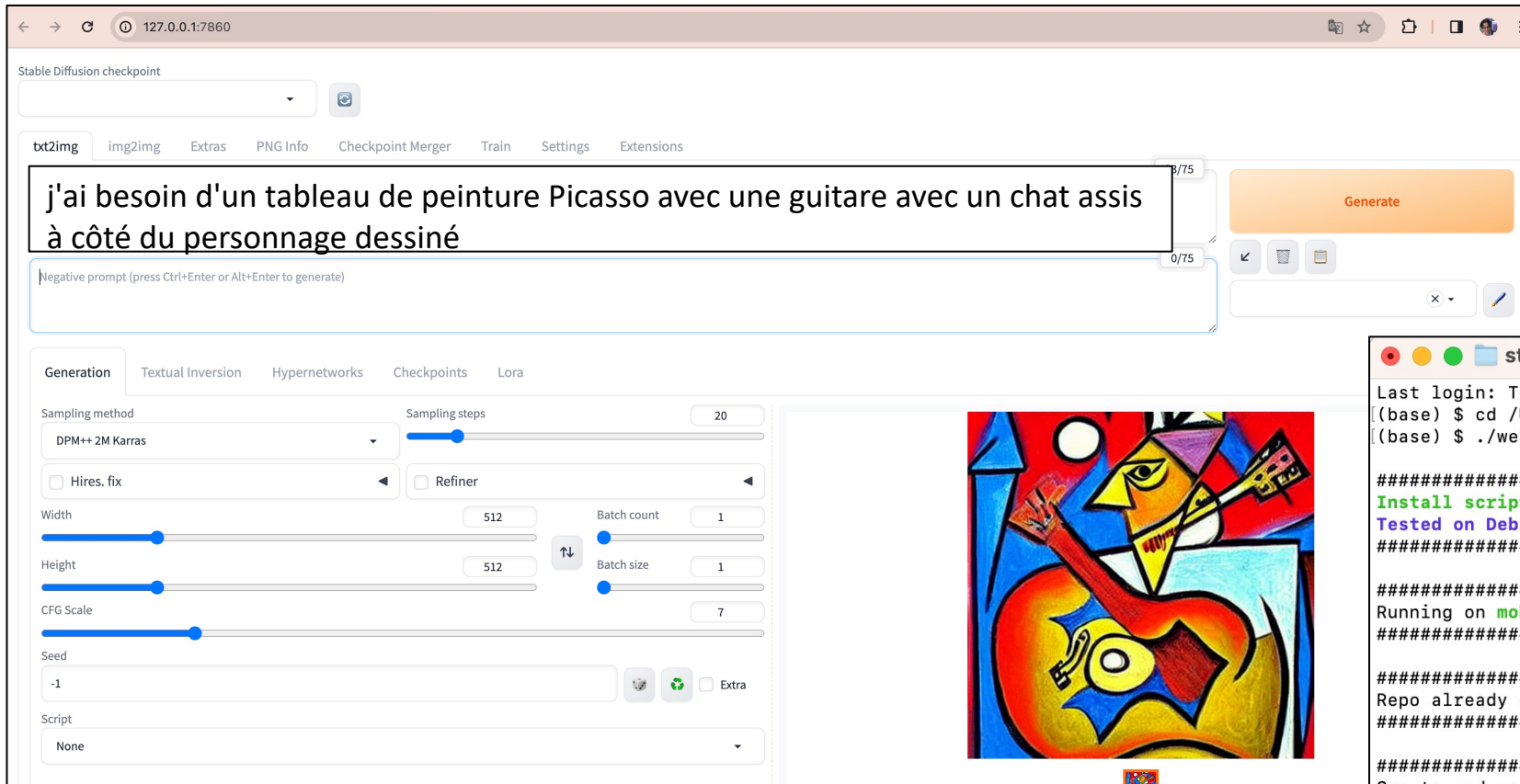
**Main Chat Area:**

- HuggingChat v0.7.0** (with a smiley face icon).
- A tagline: "Making the community's best AI chat models available to everyone."
- Current Model:** `codellama/CodeLlama-70b-Instruct-hf` (with a settings gear icon).
- Links: [Model page](#) and [Website](#).
- Examples:**
  - Fibonacci in Python
  - JavaScript promises
  - Ask anything (with a send arrow icon)
- Footer: "Model: codellama/CodeLlama-70b-Instruct-hf · Generated content may be inaccurate or false."

# Demos :

# Stable Diffusion

<http://127.0.0.1:7860/>



```
stable-diffusion-webui — python3.10 ◀ webui.sh --api — 79×24
Last login: Tue Feb 13 11:34:38 on ttys005
(base) $ cd /Users/mohamedyoussefi/stable-diffusion-webui
(base) $./webui.sh --api

#####
Install script for stable-diffusion + Web UI
Tested on Debian 11 (Bullseye), Fedora 34+ and openSUSE Leap 15.4 or newer.
#####

Running on mohamedyoussefi user
#####

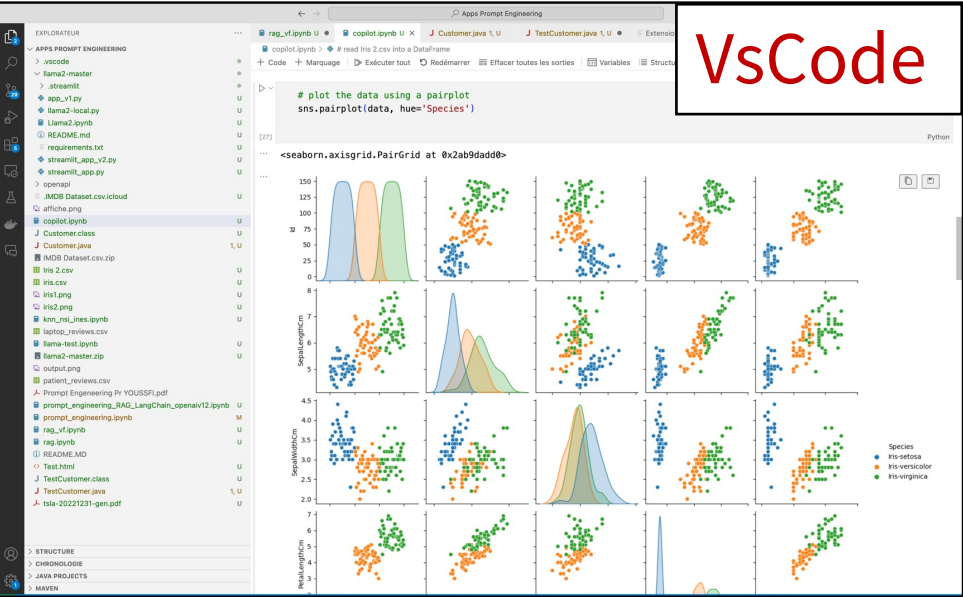
Repo already cloned, using it as install directory
#####

Create and activate python venv
#####

Launching launch.py...
#####
```

# Demos : Use Interface

## Github Copilot : Code generation



	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	\
94	95	5.6	2.7	4.2	1.3	
106	107	4.9	2.5	4.5	1.7	
141	142	6.9	3.1	5.1	2.3	
93	94	5.0	2.3	3.3	1.0	
19	20	5.1	3.8	1.5	0.3	
22	23	4.6	3.6	1.0	0.2	

	Species
94	Iris-versicolor
106	Iris-virginica
141	Iris-virginica
93	Iris-versicolor
19	Iris-setosa
22	Iris-setosa

- # import packages for machine learning and data manipulation
- # Load the data iris.csv
- # print the first 5 rows of the dataframe
- # display stats of the DataFrame
- # Remove Id column
- # display statistics of the dataframe
- # display the number of rows and columns in the dataframe
- # Specis count
- # Plot Species histogram
- # Plot data using pairplot
- # scatter plot of sepal length and sepal width
- # split data into features and target
- # split data into training and testing sets
- # create SVM model
- # train the model
- # Evaluate the model
- # create confusion matrix
- # create classification report
- # heapmap of confusion matrix
- # predict an example of iris
- # Feature importance radoom forest